

PROJECT REPORT

On

EC 16401 – VLSI SIGNAL PROCESSING

B.Tech. – Semester VI – ECE Branch

Submitted by

- 1) Timmy Aklis Suresh- 20234512**
- 2) Purshottam kumar -20234221**
- 3) Rishabh Patel-20234135**
- 4) Abhinav Kumar Maurya-20234181**
- 5) Dev Gupta-20234195**
- 6) Akash Agrahari-20234016**

Submitted To: Dr Shishir Maheshwari



The Department of Electronics and Communication Engineering

Motilal Nehru National Institute of Technology Allahabad

INDEX

S.NO	TOPIC	PAGE NO	Remarks
1.)	<i>Modeling and Simulation of FIR Filter using Distributed Arithmetic</i>	01-07	
2.)	<i>Modeling and Simulation of Pipelined FIR Filter using Distributed Arithmetic</i>	08-14	
3.)	<i>Modeling and Simulation of IIR Filter using Direct Form Architecture</i>	15-22	
4.)	<i>Modeling and Simulation of Retimed IIR Filter using Direct Form Architecture</i>	23-31	
5.)	<i>Modeling and Simulation of Full Adder using Verilog HDL based on Combinational Logic Design</i>	32-39	
6.)	<i>Modeling and Simulation of Pipelined Full Adder using Verilog HDL using Pipelining Technique</i>	40-48	

Modeling and simulation of FIR Filter using Distributed arithmetic:

• Introduction:

- The first part of the project involves the design and implementation of a 4-tap Finite Impulse Response (FIR) filter on FPGA.
- The FIR filter is based on the convolution equation

$$y(n) = b_0x(n) + b_1x(n-1) + b_2x(n-2) + b_3x(n-3),$$

where the chosen filter coefficients are 2, -4, 1, and 2.

- To achieve efficient hardware implementation, the filter is realized using the **Distributed Arithmetic (DA) algorithm**, which replaces multipliers with lookup tables and shift-add operations, thereby reducing hardware complexity.

• Distributed Arithmetic:

- Finite Impulse Response (FIR) filters are widely used in digital signal processing applications such as communication and video systems, where high performance in terms of speed, area, and power consumption is essential.
- An FIR filter operates based on the convolution of the input signal with its impulse response, mathematically expressed as

$$y(n) = \sum b_m x(n-m),$$

where the output is obtained by multiplying input samples with fixed coefficients and summing the results. However, this operation involves multiple multiplications, which are hardware intensive.

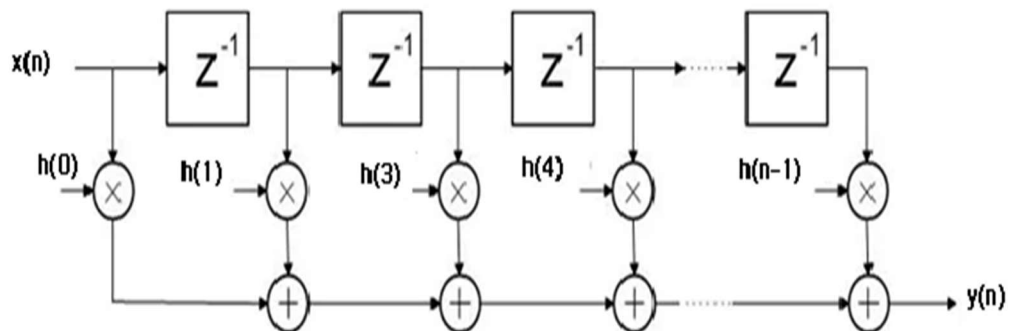


Fig 1.1) Basic FIR Filter Structure

- The Distributed Arithmetic (DA) algorithm is an efficient technique for implementing FIR filters by replacing multipliers with lookup tables (LUTs) and shift-accumulate operations.
- Since FIR filter coefficients are fixed, the multiplication terms are constant and repetitive, allowing them to be precomputed and stored in LUTs.

- This eliminates the need for hardware multipliers, significantly reducing area and complexity.
- As a result, DA is highly suitable for FPGA implementations, offering an efficient and resource-optimized design.

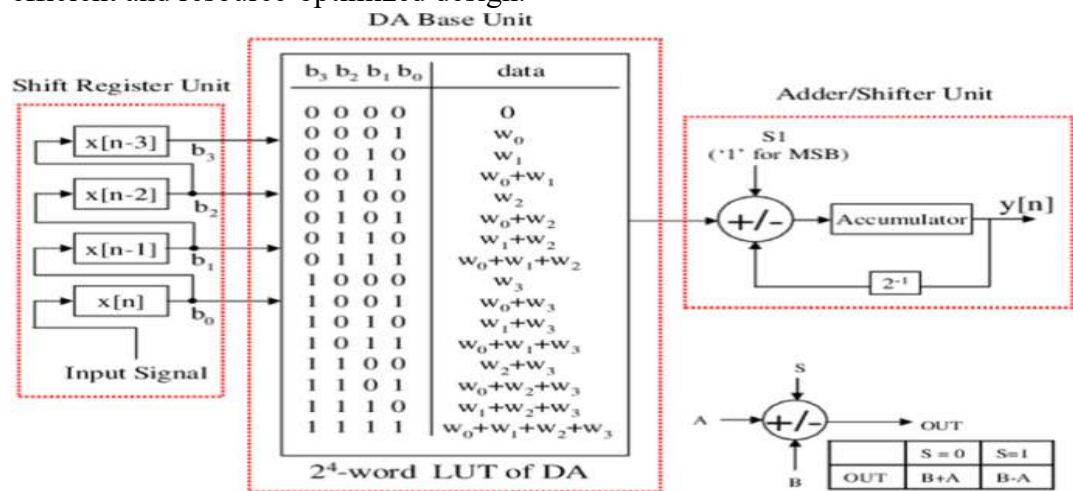


Fig 1.2) Distributed Arithmetic implementation of Fir Filter

• **Simulation of the proposed 4tap Fir filter:**

- The simulation of the proposed 4-tap FIR filter is performed to verify its functional correctness and analyze its behavior before hardware implementation.
- In simulation, a testbench is used to apply a sequence of input samples to the filter, and the corresponding output is observed through waveform analysis.
- The waveform clearly shows the relationship between the input signal and the filtered output, demonstrating the convolution operation of the FIR filter.
- Initially, the output may show transient behavior due to pipeline filling or initial conditions, after which it stabilizes and reflects the expected filtered response.
- Simulation also helps in validating the timing of operations and ensuring that the design functions correctly for different input patterns.
- This step is essential for confirming the correctness of the design and for further analysis such as switching activity used in power estimation.

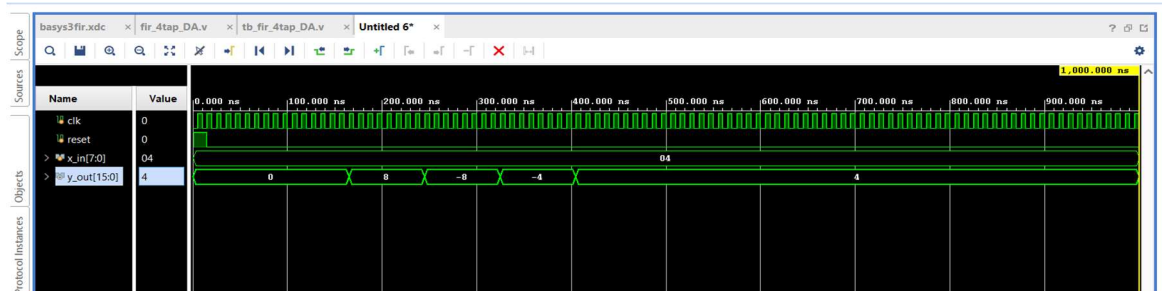


Fig 1.3) Simulation Results of 4tap FIR Filter

- **Hardware Implementation of the proposed 4tap Fir filter on BASYS3-FPGA Kit:**

- The proposed 4-tap FIR filter is implemented on the Basys 3 FPGA Kit, a widely used FPGA development board based on the Xilinx Artix-7 (XC7A35T) device.

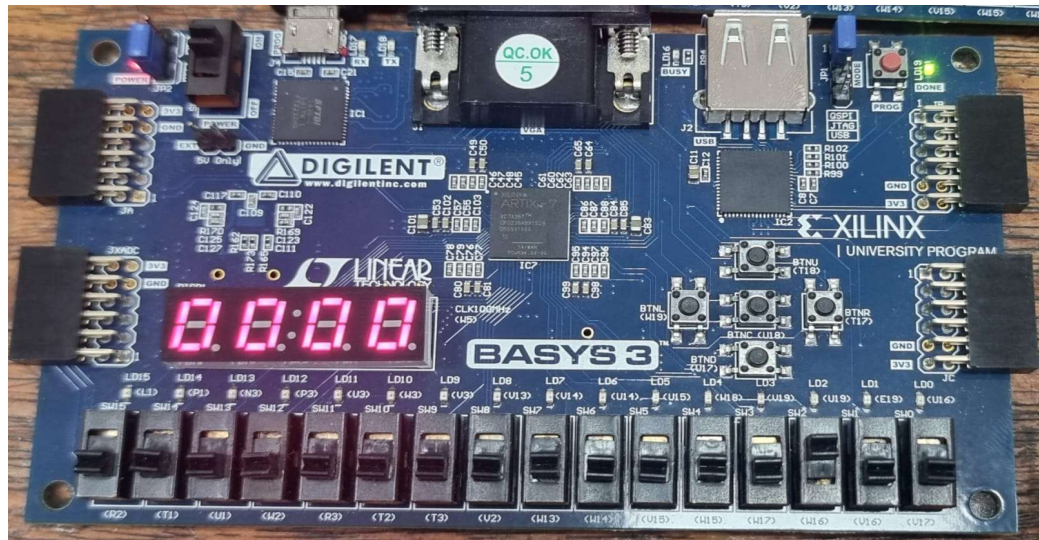


Fig 1.4) A Basys 3 FPGA Kit

- The board is designed for digital system design and prototyping and offers a range of onboard peripherals, including slide switches, push buttons, LEDs, and a four-digit 7-segment display. It also features an onboard 100 MHz clock source and USB-based programming interface, making it convenient for implementing and testing real-time hardware designs.
 - The observed hardware output follows the expected transient behavior of the FIR filter. For a constant input, the output sequence progresses as 0, 8, -8, -4, and finally settles at 4 after the initial cycles.
 - However, since the display system cannot represent negative numbers directly and operates using unsigned representation, negative values are displayed in their wrapped form based on 8-bit representation. Specifically, -8 appears as 248 and -4 appears as 252, which correspond to their two complement equivalents (i.e., values obtained by subtracting from 256).
- **Output Results of 4tap FIR Filter:**
 - The output of the implemented 4-tap FIR filter is observed directly on the hardware using the onboard display of the Basys 3 FPGA Kit.
 - The results confirm the correct operation of the filter. For a constant input, the output initially exhibits transient values due to the sequential loading of input samples into the filter stages and then settles to a steady-state value.
 - The observed sequence of outputs follows the expected FIR filter response, thereby validating the correctness of the hardware implementation.

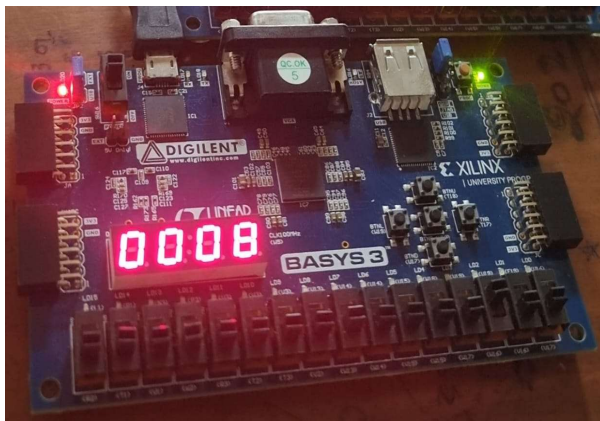
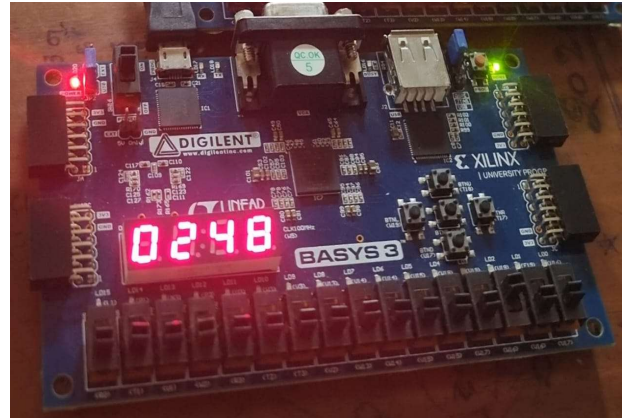
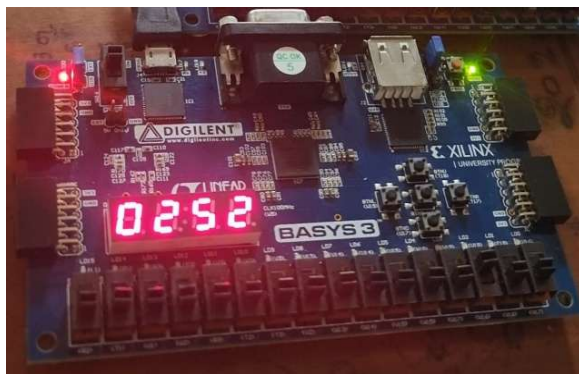


Fig 1.5) First Output of a 4 tap FIR filter



**Fig 1.6) Second Output of a 4 tap FIR filter
(wrap around effect)**



**Fig 1.7) Third Output of a 4 tap FIR filter (wrap around effect)
filter**



Fig 1.8) Fourth Output of a 4 tap FIR

- **Timing Analysis of the 4-Tap FIR Filter:**

- The timing analysis of the implemented design is carried out using the Vivado timing summary report, which provides key parameters such as Worst Negative Slack (WNS), Worst Hold Slack (WHS), and Worst Pulse Width Slack (WPWS).
- These parameters indicate whether the design meets the required timing constraints.
- A positive slack value confirms that there are no timing violations.
- The **critical path delay** of the design can be estimated by subtracting the WNS from the specified clock period, which gives an indication of the maximum delay in the design.

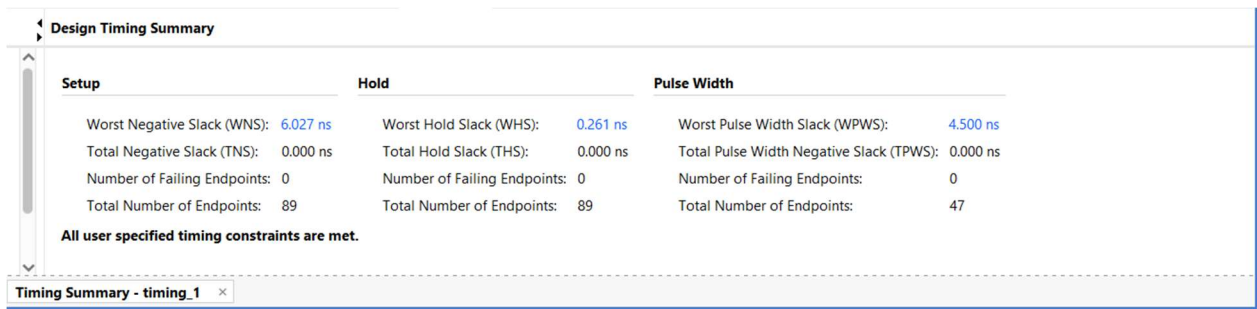


Fig 1.9) Design Timing Summary of 4 tap FIR filter

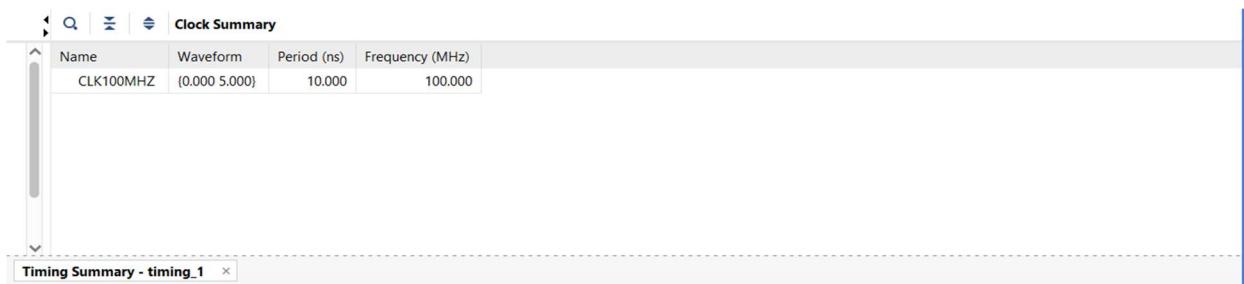


Fig 1.10) Clock Period of 4 tap FIR filter

Critical Path delay = Clock Period-WNS=10ns-6.027ns=3.973 ns

Frequency = $1/T_{cp}=1/3.973ns=251.6$ MHz

- Resource Utilization of the 4-Tap FIR Filter:**

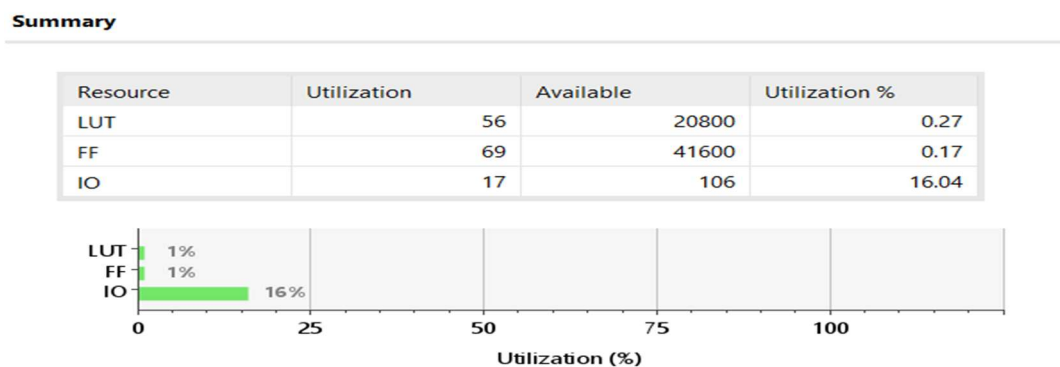


Fig 1.11) Overall Resource Utilization Summary of the Design

Hierarchy						
Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
N top_basys3	56	69	31	56	17	1

Fig 1.12) Hierarchical Resource Utilization of the Top Module

• **Power Analysis of the 4-Tap FIR Filter:**

- The power analysis of the implemented 4-tap FIR filter is carried out using the Vivado power report tool.
- The analysis provides an estimation of both static and dynamic power consumption of the design.
- Static power is mainly due to leakage currents in the FPGA, while dynamic power is caused by switching activity in clocks, logic, and signals.
- The results indicate that the total on-chip power consumption is low, demonstrating that the design is power-efficient and suitable for FPGA-based implementations.

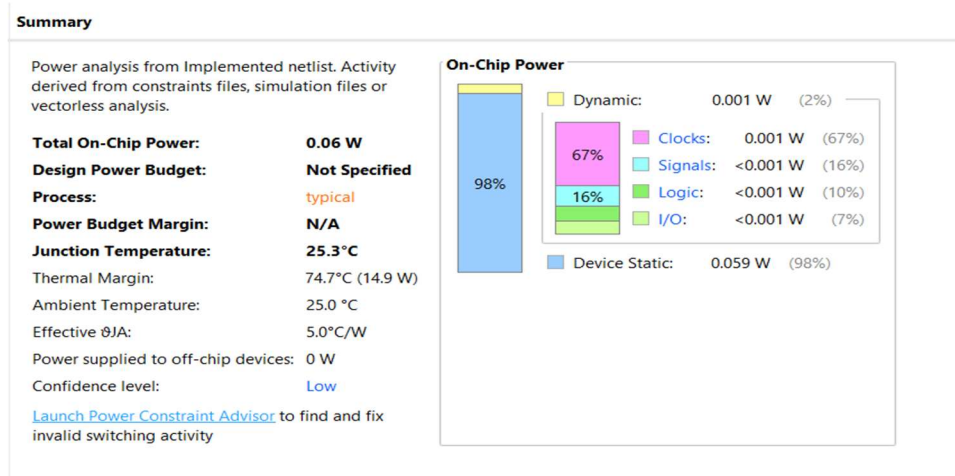


Fig 1.13) On-Chip Power Distribution of the FIR Filter

Settings	Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Summary (0.06 W, Margin: N/A)	Vccint	0.950	0.007	0.001	0.006	Unspecified	NA
Power Supply	Vccaux	1.800	0.011	0.000	0.011	Unspecified	NA
Utilization Details	Vcco33	3.300	0.000	0.000	0.000	Unspecified	NA
Hierarchical (0.001 W)	Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Clocks (0.001 W)	Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Signals (<0.001 W)	Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Data (<0.001 W)	Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Set/Reset (<0.001 W)	Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Logic (<0.001 W)	Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
I/O (<0.001 W)	Vccbram	0.950	0.000	0.000	0.000	Unspecified	NA
	MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
	Vccadc	1.800	0.018	0.000	0.018	Unspecified	NA

Fig 1.14) Power Supply Breakdown of the FIR Filter Design

• **Elaborated Design of the Hardware Circuit of 4-Tap FIR Filter:**

- The hardware design of the 4-tap FIR filter is based on the convolution operation using current and delayed input samples.
- The design includes shift registers for storing past inputs and combinational logic for computing the output.
- In this implementation, Distributed Arithmetic (DA) is used to replace multipliers with LUT-based and shift-accumulate operations, resulting in a compact and efficient FPGA design.
- The overall architecture is illustrated in the block diagram below.

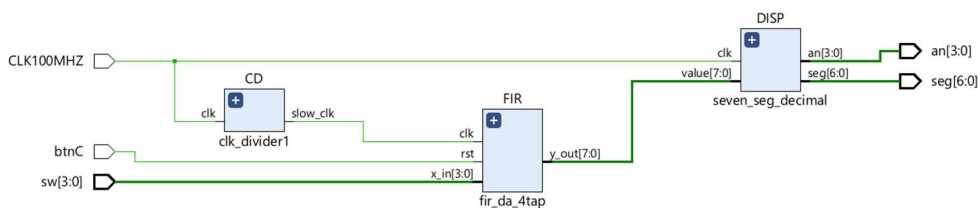


Fig 1.15) Elaborated Design of the FIR Filter Design

- **Conclusion:**

- The 4-tap FIR filter has been successfully designed and implemented on the FPGA using the Distributed Arithmetic technique.
- The results obtained from hardware and analysis confirm the correct functionality of the design along with proper resource utilization and timing performance.
- This implementation serves as a baseline for further optimization, such as pipelining, which can improve the overall performance of the system.

1.) **Modeling and Simulation of Pipelined FIR Filter using Distributed Arithmetic:**

- **Introduction:**

- The pipelined 4-tap FIR filter is designed to improve the performance of the basic FIR filter by introducing pipeline stages in the computation. The filter is based on the equation

$$y(n) = 2x(n) - 4x(n - 1) + x(n - 2) + 2x(n - 3),$$

where the coefficients are 2, -4, 1, and 2.

- In this implementation, a 4-level pipelining approach is used, where the overall computation is divided into four stages using the Distributed Arithmetic (DA) technique.
- Due to these pipeline stages, the filter introduces an initial latency of four clock cycles, after which valid outputs are produced at every clock cycle.
- This approach reduces the critical path delay and improves the overall processing speed of the system.

- **Simulation of the proposed 4tap Pipelined Fir filter:**

- The waveform shows that, due to the presence of four pipeline stages, the output is delayed by four clock cycles during the initial phase, known as pipeline filling.
- After this initial latency, the filter produces valid outputs at every clock cycle.
- The observed output sequence matches that of the non-pipelined FIR filter, confirming that pipelining does not alter the filter response but improves the processing speed by reducing the critical path delay.

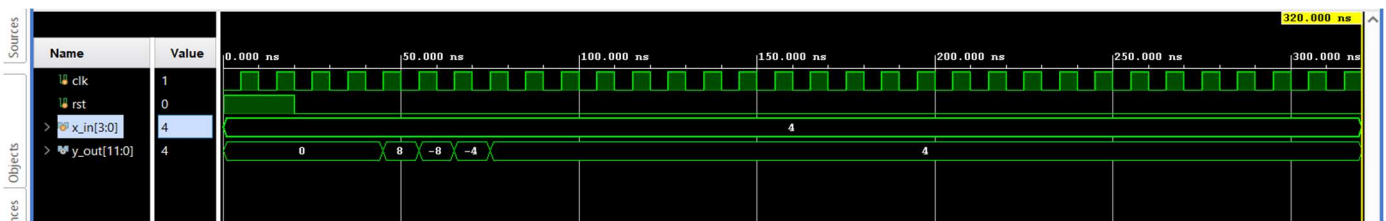


Fig 2.1) Simulation Results of 4tap Pipelined FIR Filter

- **Output Results of 4 tap Pipelined FIR Filter:**

- Due to the presence of four pipeline stages, the output exhibits an initial delay of four clock cycles (pipeline latency).
- During the initial cycles, the output remains zero as the pipeline is being filled.
- After the pipeline is filled, the filter produces valid outputs at every clock cycle.
- The output sequence follows the same pattern as the non-pipelined FIR filter, confirming functional correctness.
- For a constant input, the output initially shows transient values and then settles to a steady-state value.
- Negative output values are displayed in wrapped form due to unsigned representation on the display.

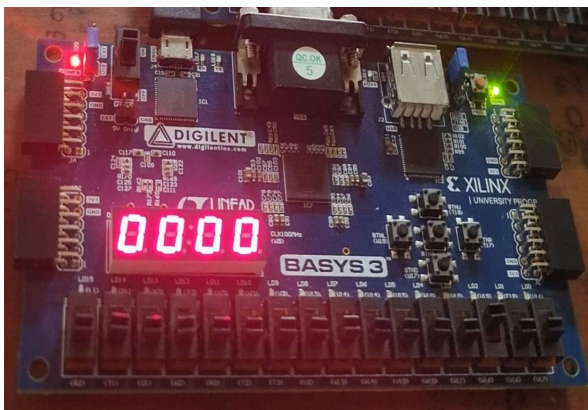


Fig 2.2) Output During Initial Pipeline Filter at

(First 4 Clock Cycles)

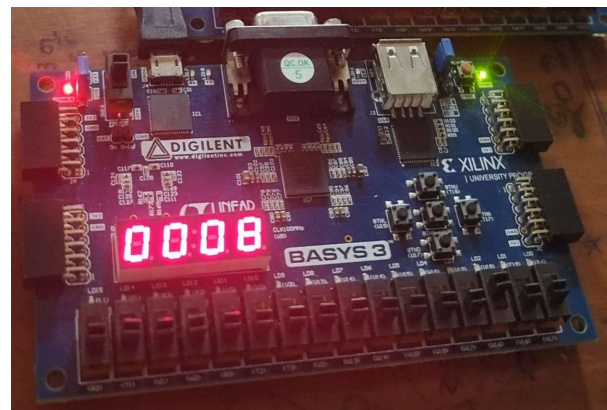


Fig 2.3) First Valid Output of the Pipelined FIR

the 5th Clock Cycle

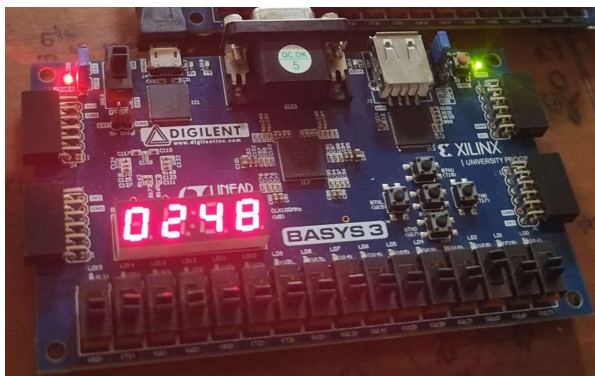


Fig 2.4) Second Valid Output at 6th Clock Cycle Clock

(wrap around effect)

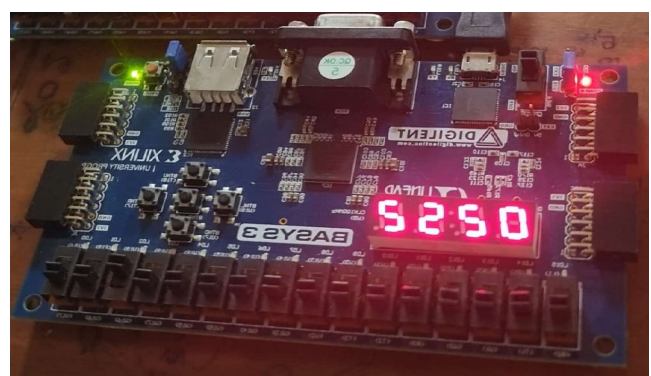


Fig 2.5) Third Valid Output of the Filter at the 7th

Cycle (wrap around effect)

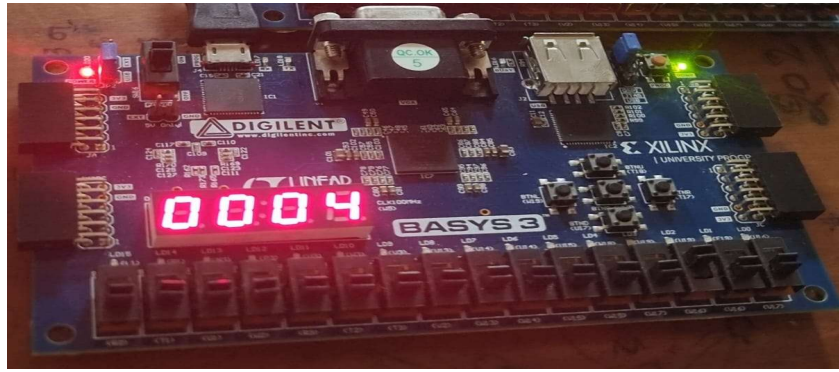


Fig 2.6) Fourth Valid Output of the Pipelined FIR Filter
at the 8th Clock Cycle

- **Timing Analysis of the 4-Tap Pipelined FIR Filter:**

- The timing analysis is performed using the Vivado timing summary report.
- Key parameters such as Worst Negative Slack (WNS), Worst Hold Slack (WHS), and Worst Pulse Width Slack (WPWS) are analyzed.
- The design shows positive slack values, indicating that there are no timing violations.
- Due to pipelining, the critical path delay is reduced compared to the non-pipelined FIR filter.
- The reduced critical path enables the design to operate at a higher clock frequency.
- The introduction of pipeline stages improves overall timing performance while maintaining correct functionality.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.657 ns	Worst Hold Slack (WHS): 0.096 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 89	Total Number of Endpoints: 89	Total Number of Endpoints: 47
All user specified timing constraints are met.		

Critical Path delay = Clock Period-WNS=10ns-6.657ns=3.343 ns

Frequency = 1/Tcp=1/3.343ns=299.13 MHz

- **Resource Utilization of the 4-Tap Pipelined FIR Filter:**

- The resource utilization of the pipelined 4-tap FIR filter is analyzed using the Vivado synthesis report, which provides details of FPGA resources such as lookup tables (LUTs) and slice registers.
- Due to the introduction of pipeline stages, the number of registers used in the design increases compared to the non-pipelined FIR filter, as additional storage elements are required between stages.
- However, the LUT utilization remains comparable since the core computation is unchanged.
- This trade-off between increased register usage and improved performance demonstrates an efficient design suitable for high-speed FPGA-based implementations.

Summary

Resource	Utilization	Available	Utilization %
LUT	53	20800	0.25
FF	96	41600	0.23
IO	17	106	16.04

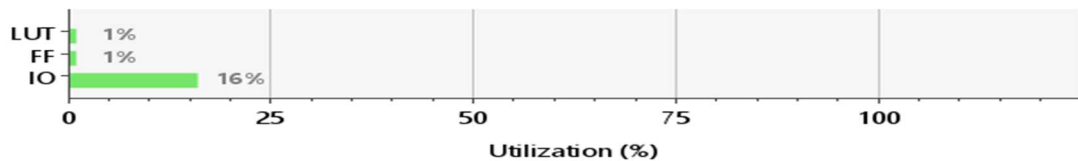
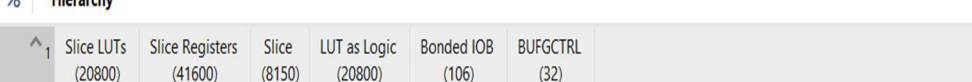


Fig 2.9) Overall Resource Utilization Summary of the Design



The screenshot shows the Vivado Hierarchy tool interface. The 'Hierarchy' tab is selected, and the 'top_basys3' module is expanded. The table below displays the resource utilization for this module.

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
> top_basys3	53	96	34	53	17	2

Fig 2.10) Hierarchical Resource Utilization of the Top Module

- **Power Analysis of the 4-Tap Pipelined FIR Filter:**

- The power analysis of the pipelined 4-tap FIR filter is performed using the Vivado power report.
- The results include both static and dynamic power consumption of the design.
- Due to the presence of additional pipeline registers, there is a slight increase in dynamic power consumption compared to the non-pipelined FIR filter, as more switching activity occurs.
- However, the improved timing performance allows the design to operate more efficiently at higher speeds.
- Overall, the design demonstrates a trade-off between power consumption and performance.

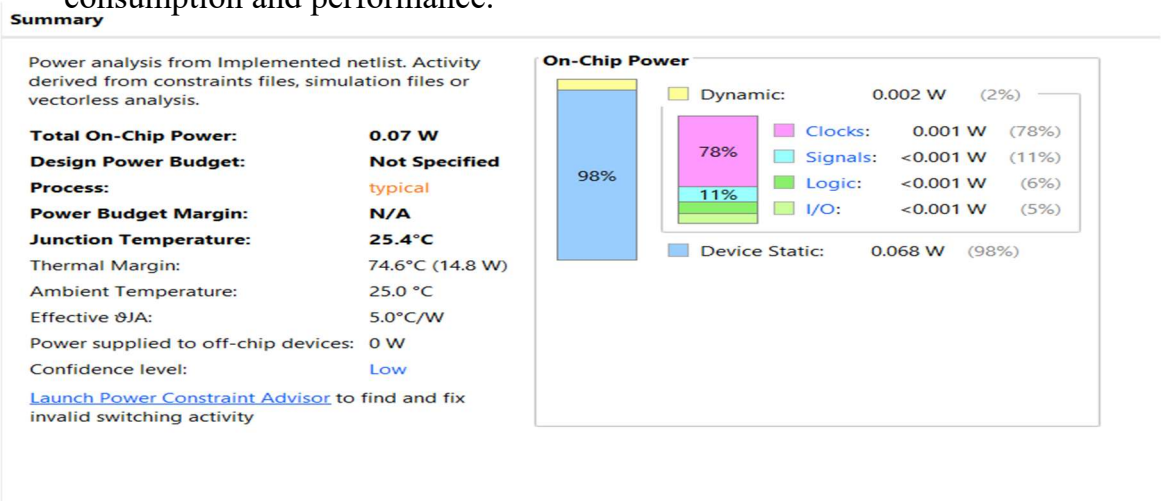


Fig 2.11) On-Chip Power Distribution of the Pipelined FIR

Filter

Settings	Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Summary (0.07 W, Margin: N/A)	Vccint	1.000	0.011	0.002	0.010	Unspecified	NA
Power Supply	Vccaux	1.800	0.013	0.000	0.013	Unspecified	NA
Utilization Details	Vcco33	3.300	0.000	0.000	0.000	Unspecified	NA
Hierarchical (0.002 W)	Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Clocks (0.001 W)	Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Signals (<0.001 W)	Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Data (<0.001 W)	Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Set/Reset (<0.001 W)	Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Logic (<0.001 W)	Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
I/O (<0.001 W)	Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
	Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

Fig 2.12) Power Supply Breakdown of the Pipelined FIR Filter

Design

- **Elaborated Design of the Hardware Circuit of 4-Tap Pipelined FIR Filter:**

- The hardware design of the pipelined 4-tap FIR filter is based on the standard FIR structure, where input samples are delayed using shift registers and combined using fixed coefficients.
- In this implementation, the computation is divided into four pipeline stages using the Distributed Arithmetic (DA) technique.
- Each stage performs a part of the overall calculation, with intermediate results stored in pipeline registers.
- This staged processing reduces the critical path delay and improves the speed of operation.
- The overall architecture, including input registers, LUT-based computation, and pipeline stages, is shown in the block diagram below.

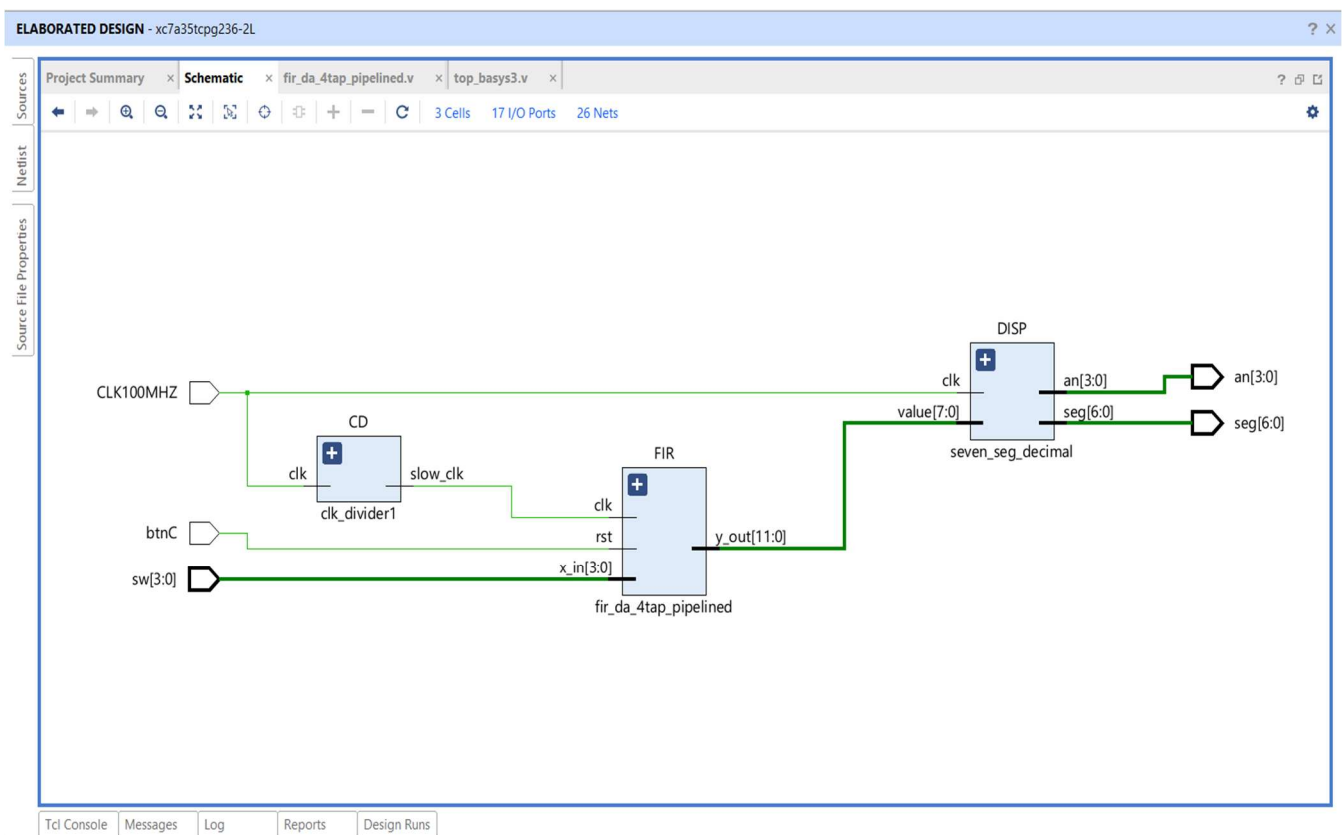


Fig 2.13) Elaborated Design of the Pipelined FIR Filter Design

- **Comparison Between Non-Pipelined and Pipelined 4-Tap FIR Filter:**

Parameter	FIR Filter	Pipelined FIR Filter
Critical Path Delay	3.973 ns (<i>higher</i>)	3.343 ns (<i>lower</i>)
Max Frequency	253 MHz (<i>lower</i>)	299 MHz (<i>higher</i>)
Latency	1 cycle (<i>lower</i>)	4 cycles (<i>higher</i>)
Throughput	1 output/cycle	1 output/cycle
LUT Utilization	56	53
SLICE registers	69 (<i>lower</i>)	96 (<i>higher</i>)
Dynamic Power	1 mW (<i>lower</i>)	2 mW (<i>higher</i>)

- **Conclusion:**

- In this project, a 4-tap pipelined FIR filter was successfully designed and implemented on the FPGA using the Distributed Arithmetic technique.
- Both non-pipelined and pipelined architectures were developed and analyzed.
- The results confirm that the pipelined FIR filter improves performance by reducing the critical path delay and increasing the maximum operating frequency.
- However, this improvement comes at the cost of increased latency and slightly higher power and register utilization.
- The comparison clearly demonstrates the trade-off between speed, power, and hardware resources.
- Overall, the pipelined design proves to be more suitable for high-speed digital signal processing applications.

2.) Modeling and Simulation of IIR Filter using Direct Form Implementation:

• Introduction:

- An Infinite Impulse Response (IIR) filter is a type of digital filter in which the output depends on both current and past input samples as well as previous output values.
- Unlike FIR filters, IIR filters use feedback, which makes them recursive in nature.
- The general form of an IIR filter is given by:

$$y(n) = \sum b_k x(n - k) + \sum a_k y(n - k)$$

- In this project, the implemented IIR filter follows the equation:

$$y(n) = 2x(n) + 2x(n - 1) + 2x(n - 2) + 2x(n - 3) + y(n - 1)$$

- The presence of the feedback term $y(n - 1)$ allows the filter to have an infinite impulse response.
- IIR filters are computationally efficient and require fewer resources compared to FIR filters for achieving similar performance.

• Direct Form I Implementation of IIR Filter:

- Direct Form I is a commonly used structure for implementing IIR filters.
- It consists of two main sections: the feedforward (FIR) part and the feedback part.
- The feedforward section processes the current and past input samples, while the feedback section uses previous output values.
- Separate delay elements are used for input samples and output samples.
- The outputs of both sections are combined using adders to produce the final output.
- This structure directly follows the mathematical equation of the IIR filter.
- It is simple to understand and easy to implement with hardware and simulation.

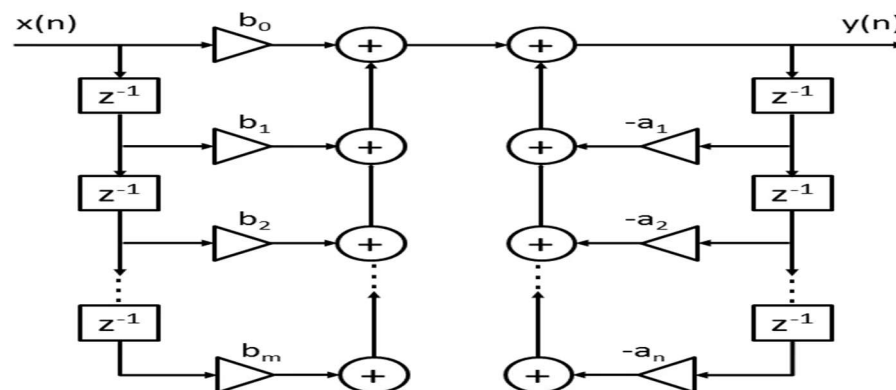


Fig 3.1) Direct Form-1 structure of IIR Filter

- **Simulation of the Proposed IIR Filter:**

- The simulation of the proposed IIR filter is carried out to verify its functional correctness.
- A constant input is applied to observe the recursive behavior of the filter.
- The waveform shows that the output depends on both current inputs and previous outputs due to the feedback term.
- Unlike FIR filters, the output does not settle immediately and continues to evolve over successive clock cycles.
- The recursive nature of the filter results in an accumulating output response.
- The simulation results match the expected theoretical output, confirming correct implementation of the IIR filter.

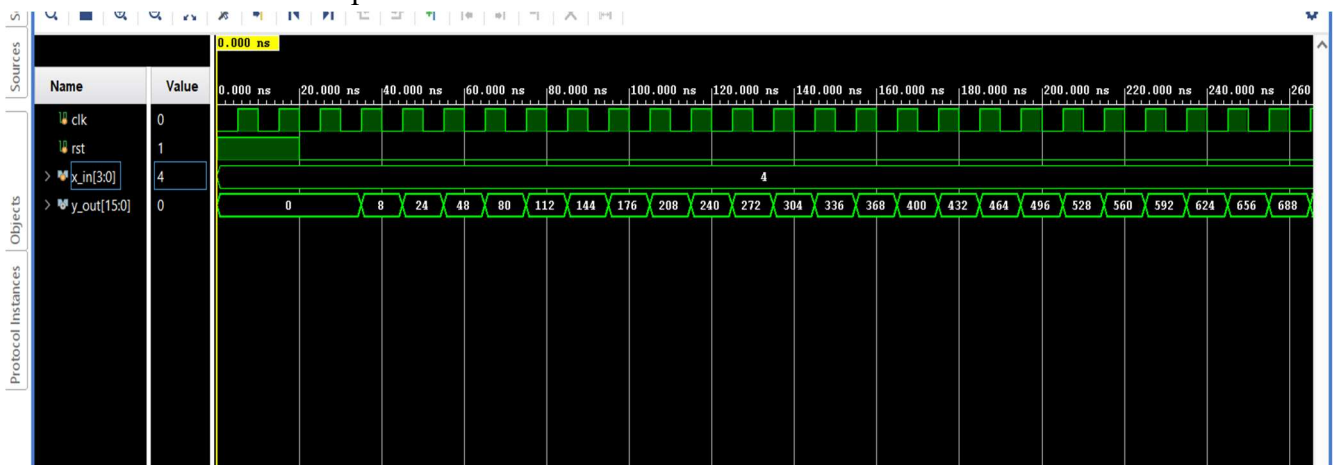


Fig 3.2) Simulation Results of IIR Filter

- **Output Results of IIR Filter:**

- The output of the IIR filter is obtained from the hardware implementation on the FPGA.
- A constant input value is applied, and the output is observed for each clock cycle.
- The output sequence shows a cumulative behavior due to the presence of the feedback term $y(n - 1)$.
- Unlike FIR filters, the output keeps increasing over successive clock cycles, demonstrating the recursive nature of the IIR filter.
- The observed output values match the expected theoretical sequence, confirming correct hardware implementation.
- It is observed that when the output exceeds the displayable range, the values wrap around due to finite bit-width representation.
- Since the display (LCD/7-segment) cannot represent large values beyond 256, the output appears in wrapped form (modulo representation).

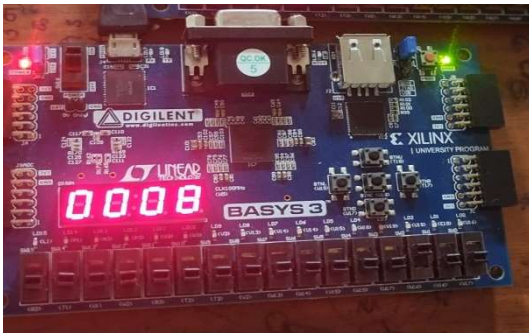


Fig 3.3) First Output of an IIR filter



Fig 3.4) Second Output of an IIR filter

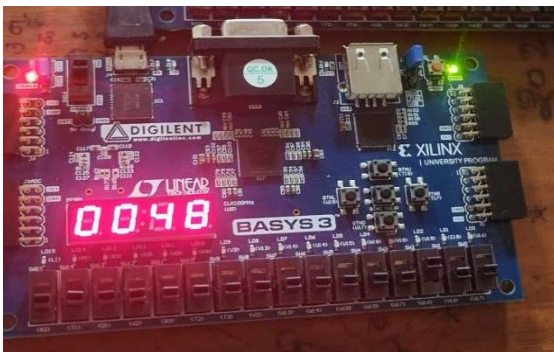


Fig 3.5) Third Output of an IIR filter

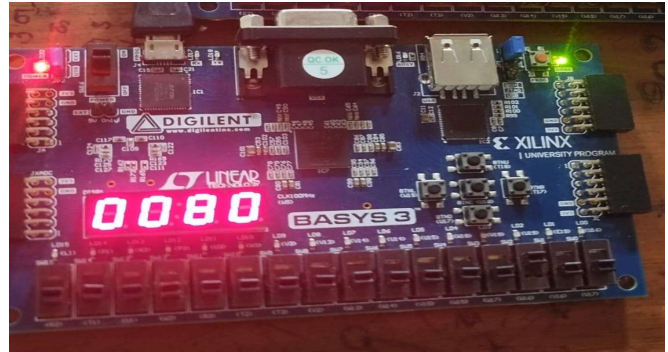


Fig 3.6) Fourth Output of an IIR filter

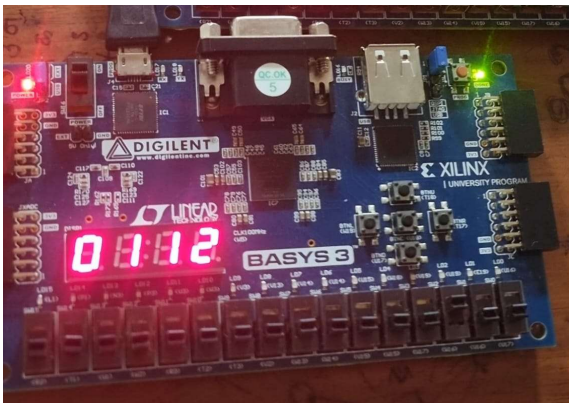


Fig 3.7) Fifth Output of an IIR filter

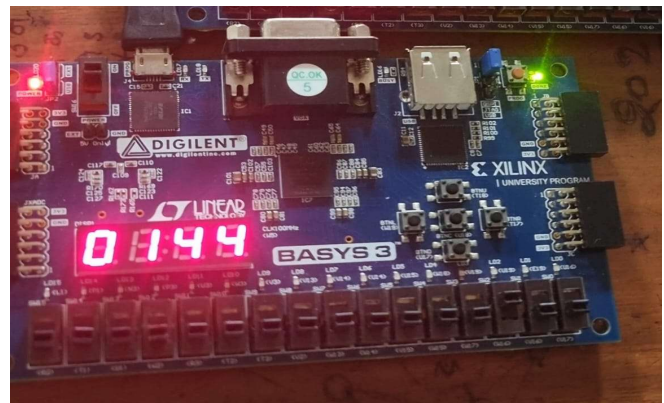


Fig 3.8) Sixth Output of an IIR filter

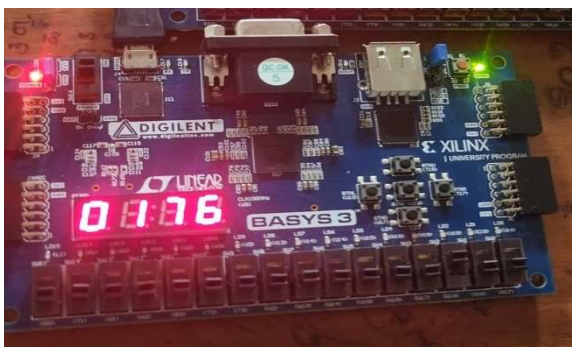


Fig 3.9) Seventh Output of an IIR filter

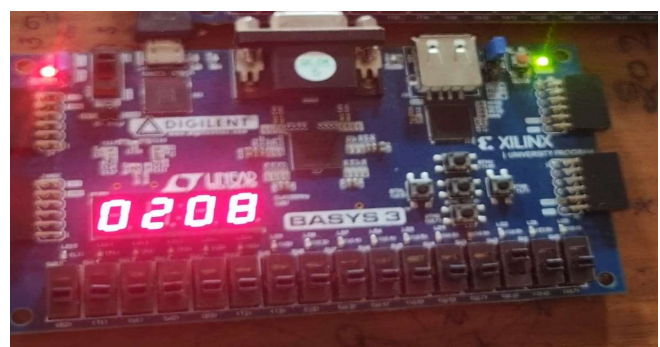


Fig 3.10) Eighth Output of an IIR filter

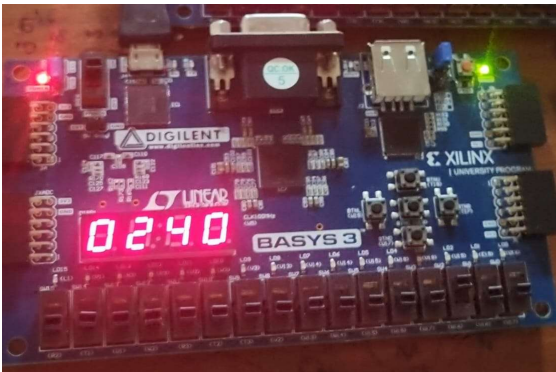


Fig 3.11) Ninth Output of an IIR filter effect)

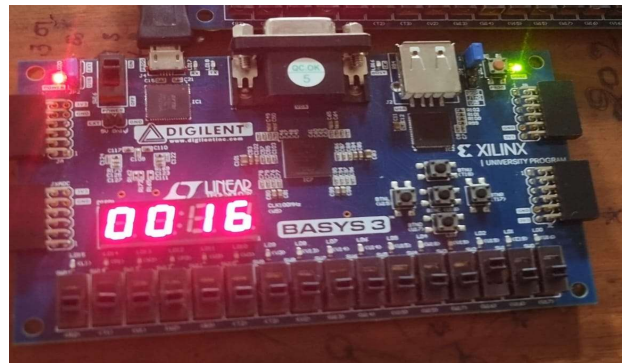


Fig 3.12) Tenth Output of an IIR filter (wrap around

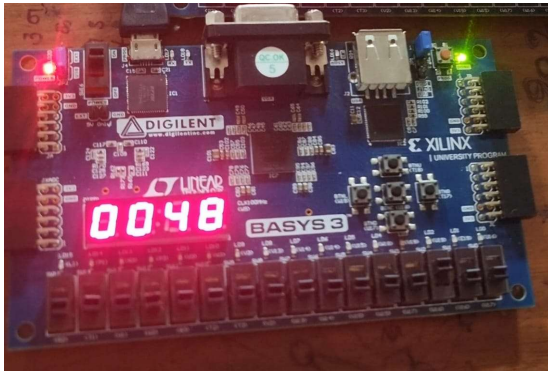


Fig 3.13) Eleventh Output of an IIR filter effect)

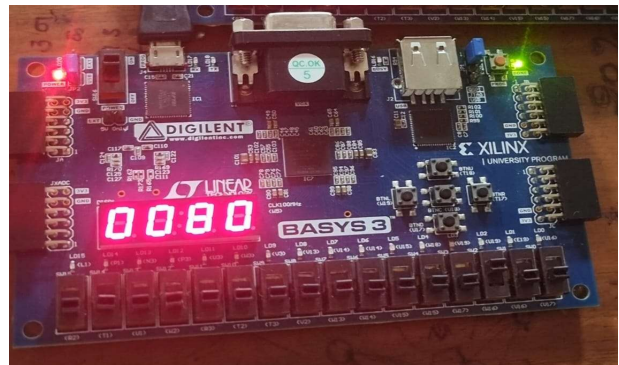


Fig 3.14) Twelfth Output of an IIR filter (wrap around

- **Timing Analysis of the IIR Filter:**

- Timing analysis is performed using the Vivado timing summary report.
- WNS, WHS, and WPWS are checked to verify timing performance.
- The feedback structure introduces a longer critical path in the design.
- No retiming is applied, so register placement is not optimized.
- This results in higher propagation delays in the combinational path.
- The design operates correctly if slack values remain positive.
- Overall timing performance is limited by the long critical path.

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.336 ns	Worst Hold Slack (WHS): 0.259 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 89	Total Number of Endpoints: 89	Total Number of Endpoints: 47
All user specified timing constraints are met.		

Fig 3.15) Design Timing Summary of IIR filter

Clock Summary			
Name	Waveform	Period (ns)	Frequency (MHz)
CLK100MHZ	{0.000 5.000}	10.000	100.000

Fig 3.16) Clock Period of IIR filter

Critical Path delay = Clock Period-WNS=10ns-6.336ns=3.664 ns

Frequency = $1/T_{cp}=1/3.664ns=272.92$ MHz

• **Resource Utilization of the IIR Filter:**

- The resource utilization of the implemented IIR filter is analyzed using the Vivado synthesis report.
- The report provides details of FPGA resources such as lookup tables (LUTs), slice registers, and other logic elements used in the design.
- LUTs are used for implementing combinational logic, while slice registers are used for storing intermediate and feedback values.
- The recursive structure of the IIR filter leads to higher utilization compared to purely feedforward architecture.
- Multipliers and adders contribute significantly to the overall hardware usage.
- The results reflect an efficient FPGA mapping of the IIR filter design.
- Overall, the design achieves correct functionality with acceptable resource consumption.

Summary

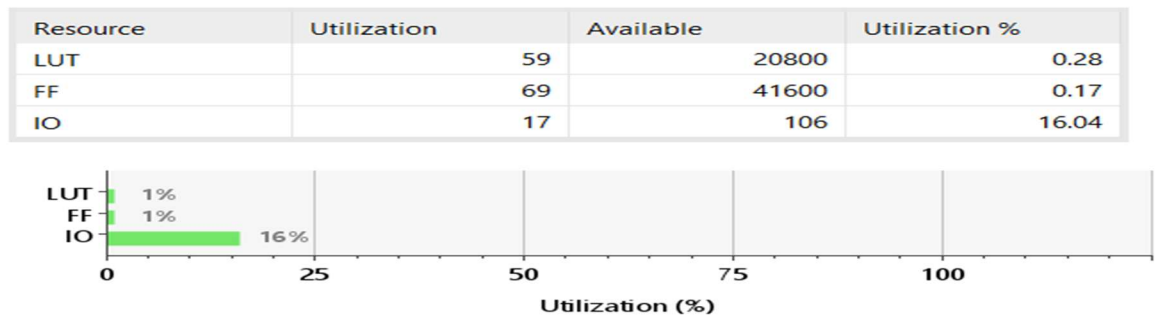


Fig 3.17) Overall Resource Utilization Summary of the Design

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
top_bsys3	59	69	34	59	17	1

Fig 3.18) Hierarchical Resource Utilization of the Top Module

• **Power Analysis of the IIR Filter:**

- The power analysis of the implemented IIR filter is carried out using the Vivado power report tool.
- The report estimates both static and dynamic power consumption of the design.
- Static power is mainly due to leakage currents in the FPGA device.
- Dynamic power arises from switching activity in logical elements, registers, and clock networks.
- The feedback nature of the IIR filter contributes to continuous signal activity within the circuit.
- The results show moderate power consumption based on the implemented architecture and operating frequency.
- Overall, the design is power-efficient and suitable for FPGA-based digital signal processing applications.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.07 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.3°C
 Thermal Margin: 74.7°C (14.8 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

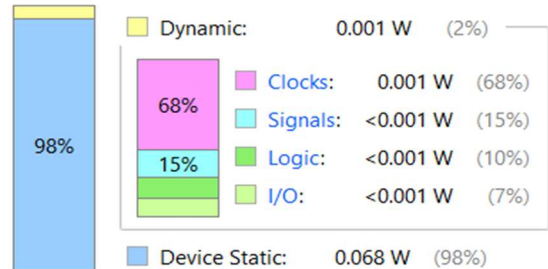


Fig 3.19) On-Chip Power Distribution of the IIR Filter

Settings	Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Summary (0.07 W, Margin: N/A)	Vccint	1.000	0.011	0.001	0.010	Unspecified	NA
Power Supply	Vccaux	1.800	0.013	0.000	0.013	Unspecified	NA
Utilization Details	Vcco33	3.300	0.000	0.000	0.000	Unspecified	NA
Hierarchical (0.001 W)	Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Clocks (0.001 W)	Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Signals (<0.001 W)	Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Data (<0.001 W)	Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Set/Reset (<0.001 W)	Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Logic (<0.001 W)	Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
I/O (<0.001 W)	Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
	Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

Fig 3.20) Power Supply Breakdown of the IIR Filter Design

- **Elaborated Design of the Hardware Circuit of IIR Filter:**

- The hardware design of the IIR filter is based on a recursive structure that uses both present input samples and previous output samples.
- The design includes delay elements (registers) to store past input and output values required for feedback operation.
- Combinational logic is used to perform multiplication and addition operations for both feedforward and feedback paths.
- The output is generated using a combination of current input and weighted past outputs, forming a feedback loop.
- The overall architecture is implemented in FPGA using adders, multipliers, and registers in a structured dataflow manner.
- The block diagram of the implemented IIR filter is shown below.

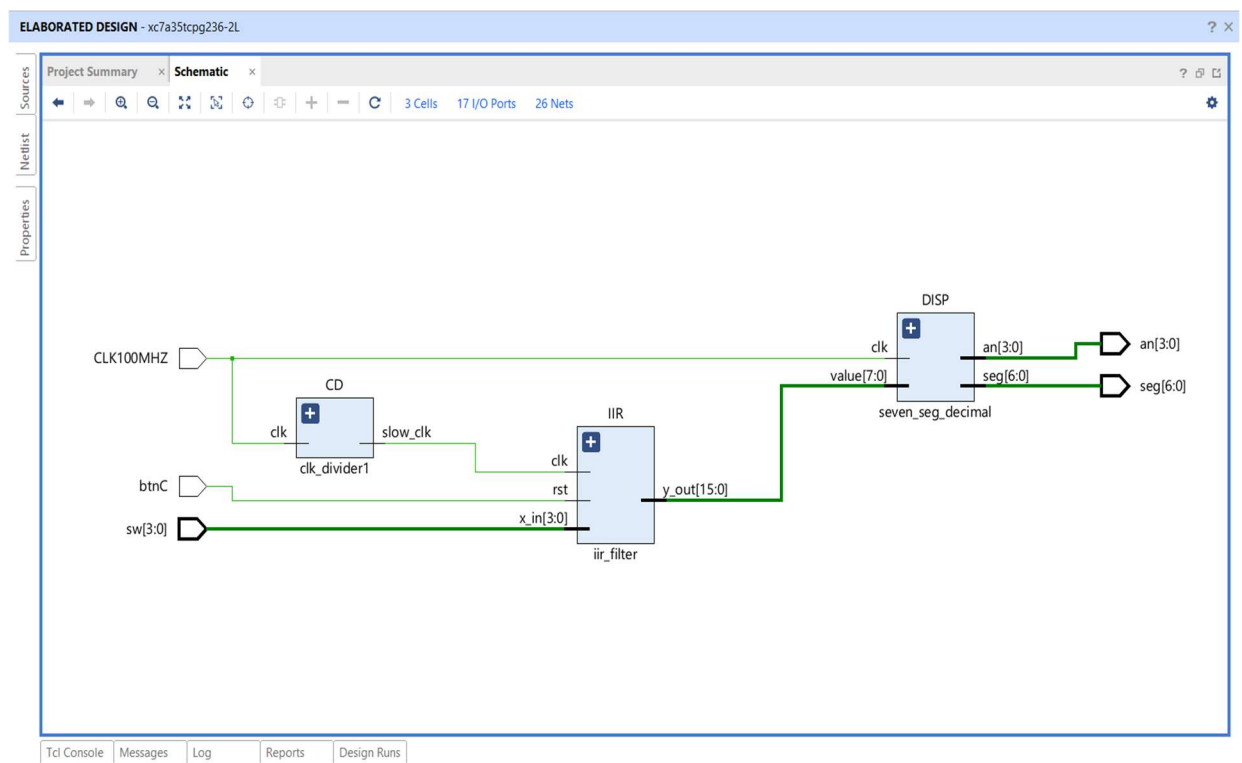


Fig 3.21) Elaborated Design of the IIR Filter Design

- **Conclusion:**

- The IIR filter has been successfully designed and implemented on the FPGA using recursive (feedback-based) architecture.
- The results obtained from simulation and hardware analysis confirm the correct functionality of the design.
- The design meets the required performance in terms of timing, power, and resource utilization as per the synthesis reports.
- This implementation provides a foundation for further optimizations such as retiming or pipelining to enhance performance

3.) **Modeling and Simulation of Retimed IIR Filter using Direct Form Implementation:**

- **Introduction:**

- The 4-tap IIR filter is designed to improve speed and timing performance of a basic recursive filter.
- The filter consists of both feedback components.
- The difference equation used is:

$$y(n)=2x(n)+2x(n-1) +2x(n-2) +2x(n-3) +y(n-1)$$

- The feedforward coefficients are 2, and the feedback coefficient is 1.
- The output depends on both current/past input samples and the previous output sample.
- Retiming is applied by moving registers into the critical paths of the design.
- This helps in breaking long combinational delay paths in both forward and feedback sections.
- The computation is distributed across multiple clock cycles using register balancing.
- This reduces the critical path delay and improves the maximum operating frequency.
- The design introduces an initial latency of a few clock cycles.
- After the initial delay, the filter produces one output per clock cycle.
- This retimed structure improves overall hardware efficiency compared to basic IIR implementation.

- **Simulation of the proposed Retimed IIR filter:**

- The simulation of the proposed 4-tap IIR filter is performed to verify its functional correctness and timing behavior.
- The waveform shows that the output is available after an initial delay of one clock cycle due to register balancing in the design.
- This single-cycle delay represents the internal computation and storage of intermediate values in the retimed structure.

- After this initial delay, the filter produces one valid output per clock cycle.
- The output sequence matches the expected theoretical IIR response, confirming correct implementation of both feedforward and feedback paths.
- The retimed structure ensures proper timing closure by reducing long combinational paths in the recursive loop.
- This improves overall operating frequency while maintaining the correct filter functionality.

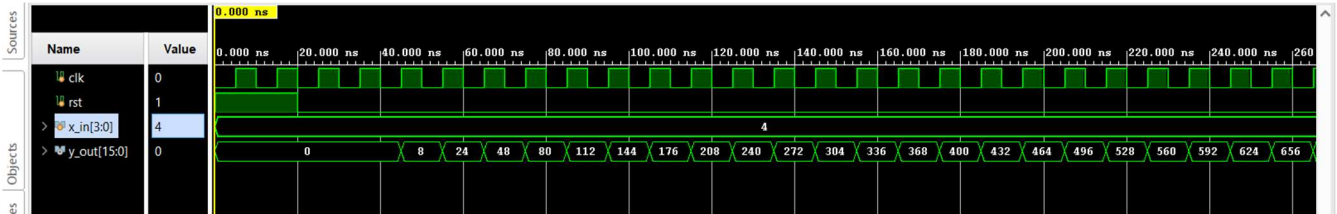


Fig 4.1) Simulation of the Retimed IIR Filter Design

• **Output Results of Retimed IIR Filter:**

- The IIR filter output is obtained from FPGA implementation with a constant input applied.
- The output appears after an initial delay of the one-clock cycle due to register balancing.
- After this, one valid output is produced every clock cycle.
- The output shows cumulative behavior due to the feedback term $y(n - 1)$.
- The response matches the expected theoretical IIR behavior, confirming correct implementation.
- Output wrapping is observed when values exceed display range due to finite bit-width representation.
- The display shows modulo behavior as it cannot represent large values beyond its limit.



Fig 4.2) Output During First Clock Cycle

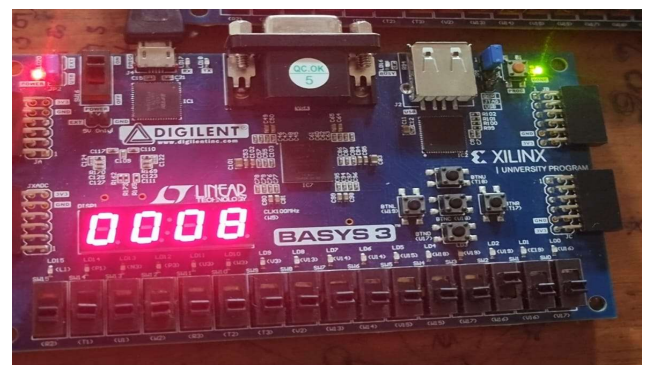


Fig 4.3) First Valid Output at the 3rd Clock



Fig 4.4) Second Valid Output at the 4th Clock Cycle

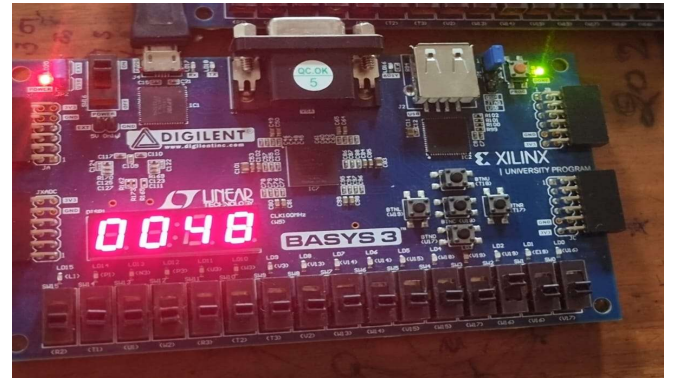


Fig 4.5) Third Valid Output of the at the 5th Clock



Fig 4.6) Fourth Valid Output at the 6th Clock Cycle

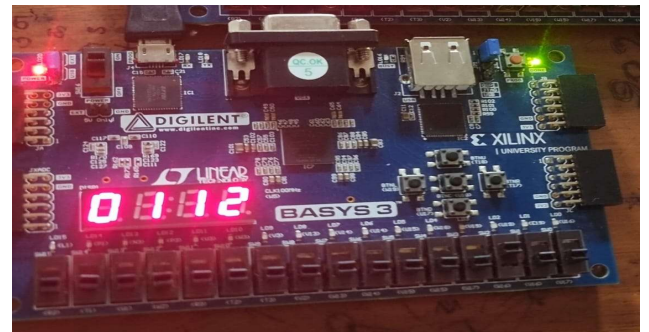


Fig 4.7) Fifth Valid Output at the 7th Clock

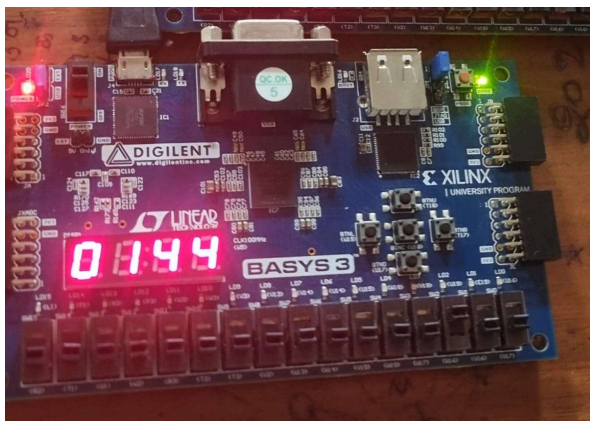


Fig 4.8) Sixth Valid Output at 8th Clock Cycle

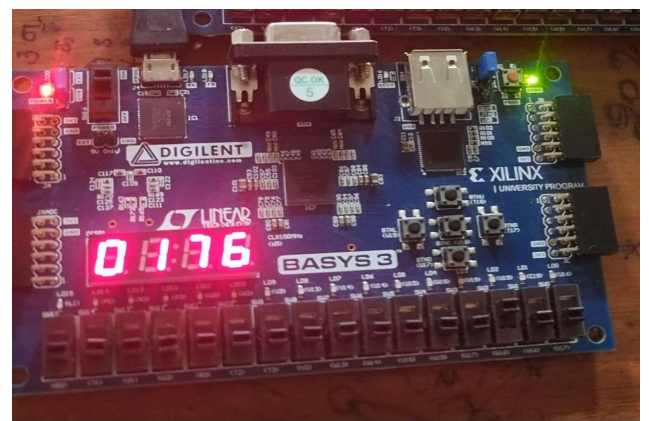


Fig 4.9) Seventh Valid Output at the 9th Clock

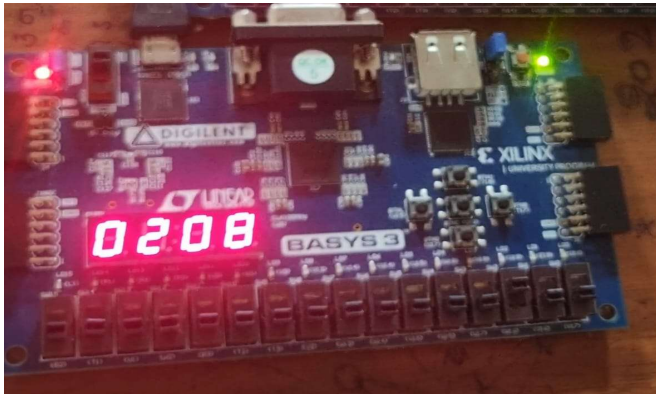


Fig 4.10) Eighth Valid at the 10th Clock Cycle

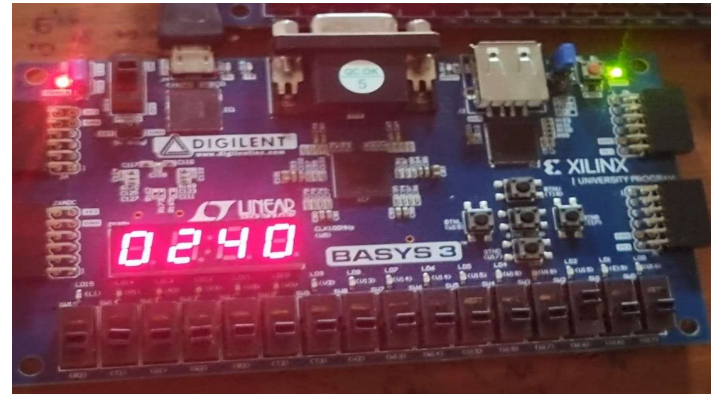


Fig 4.11) Ninth Valid Output at the 11th Clock

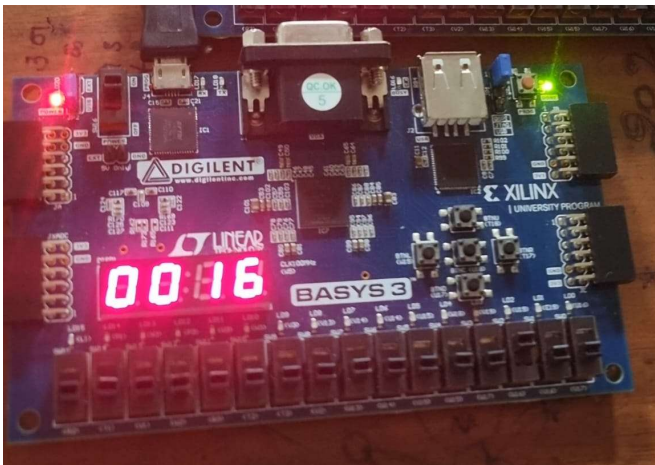


Fig 4.12) Tenth Valid Output at the 12th Clock Cycle
Cycle (wrap around effect)

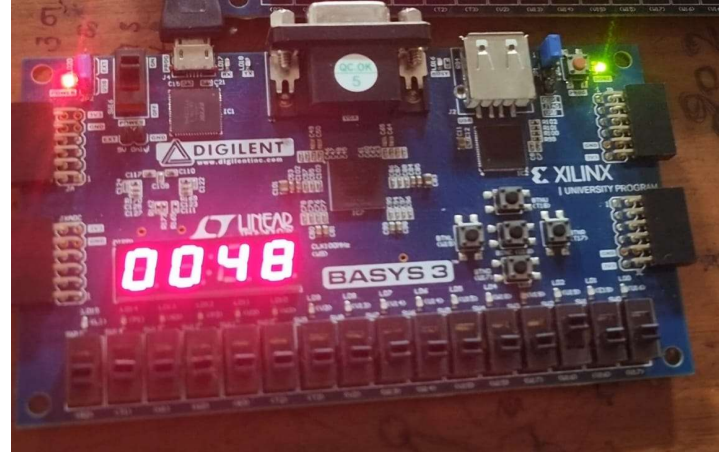


Fig 4.13) Eleventh Valid Output at the 13th Clock
Cycle (wrap around effect)

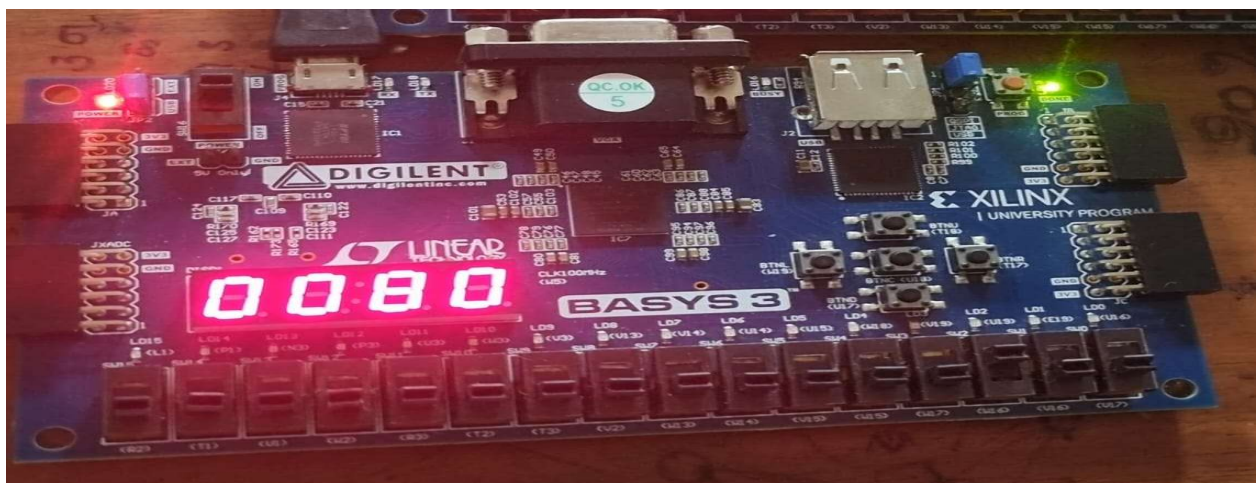


Fig 4.14) Twelfth Valid Output of the Retimed IIR Filter at the 14th Clock Cycle
(wrap around effect)

- **Timing Analysis of the Retimed IIR Filter:**

- Timing analysis is performed using the Vivado timing summary report.
- WNS, WHS, and WPWS values are checked to evaluate timing performance.
- Retiming redistributes registers to balance the feedback path.
This reduces the effective critical path delay compared to the non-retimed design.
- The design achieves improved timing slack due to optimized register placement.
- No functional change is observed in the filter response after retiming.
Overall timing performance is improved, making the design more efficient for high-speed operation.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 6.451 ns	Worst Hold Slack (WHS): 0.095 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints:	0
Total Number of Endpoints: 89	Total Number of Endpoints: 89	Total Number of Endpoints:	47
All user specified timing constraints are met.			

Fig 4.15) Design Timing Summary of Retimed IIR filter

Name	Waveform	Period (ns)	Frequency (MHz)
CLK100MHZ	{0.000 5.000}	10.000	100.000

Fig 4.16) Clock Period of Retimed IIR filter

Critical Path delay = Clock Period-WNS=10ns-6.451ns=3.549 ns

Frequency = $1/T_{cp}=1/3.549ns=281.76$ MHz

- **Resource Utilization of the Retimed IIR Filter:**

- The resource utilization of the retimed IIR filter is analyzed using the Vivado synthesis report.
- The report provides details of FPGA resources such as LUTs, slice registers, and other logic elements used in the design.
- LUTs are used for implementing combinational logic, while slice registers store intermediate and feedback values.
- Retiming redistributes registers across the design, leading to a more balanced hardware structure.
- This may slightly increase slice registers due to added pipeline balancing registers.
- The combinational logic remains unchanged, so LUT usage is nearly the same as the non-retimed design.
- Overall, the design achieves improved timing performance with minimal impact on total resource utilization.

Summary

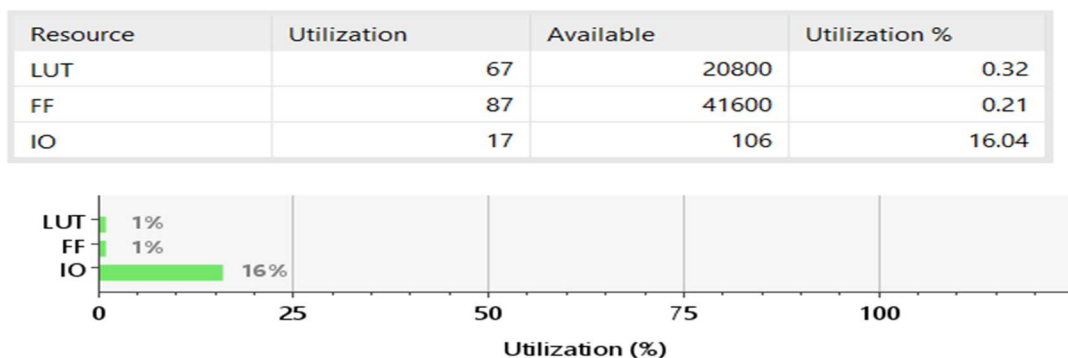


Fig 4.17) Overall Resource Utilization Summary of the Design

Hierarchy							
Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
top_basys3	67	87	6	44	67	17	2
IIR (iir_filter_retimed)	52	41	6	24	52	0	0

Fig 4.18) Hierarchical Resource Utilization of the Top Module

- **Power Analysis of the Retimed IIR Filter:**

- The power analysis of the retimed IIR filter is carried out using the Vivado power report tool.
- The report estimates both static and dynamic power consumption of the design.
- Static power is mainly due to leakage currents in the FPGA device.
- Dynamic power arises from switching activity in logic elements, registers, and clock distribution networks.
- Retiming increases register usage slightly, which can cause a small rise in clock network power.
- However, improved balancing reduces long combinational switching activity in the critical path.
- Overall, the design maintains power efficiency with a slight trade-off between timing improvement and dynamic power.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.07 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.3 °C
 Thermal Margin: 74.7 °C (14.8 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0 °C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

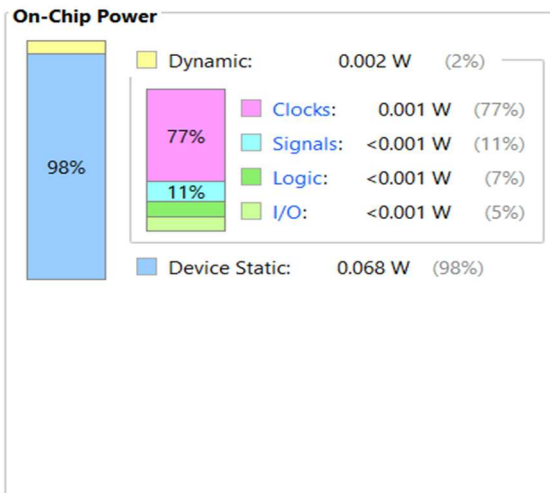


Fig 4.19) On-Chip Power Distribution of the Retimed IIR Filter

Settings	Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Summary (0.07 W, Margin: N/A)	Vccint	1.000	0.011	0.002	0.010	Unspecified	NA
Power Supply	Vccaux	1.800	0.013	0.000	0.013	Unspecified	NA
Utilization Details	Vcco33	3.300	0.000	0.000	0.000	Unspecified	NA
Hierarchical (0.002 W)	Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Clocks (0.001 W)	Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Signals (<0.001 W)	Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Data (<0.001 W)	Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Set/Reset (<0.001 W)	Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Logic (<0.001 W)	Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
I/O (<0.001 W)	Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
	Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

Fig 4.20) Power Supply Breakdown of the Retimed IIR Filter Design

- **Elaborated Design of the Hardware Circuit of Retimed IIR Filter:**

- The retimed IIR filter is based on a recursive structure that uses both current input samples and previous output samples.
- The design includes register-based delay elements to store past input and output values required for feedback operation.
- The feedforward computation is divided into multiple registered stages to reduce the combinational delay.
- Retiming is applied by relocating registers across the arithmetic path to balance the critical path.
- The output is generated by combining the computed feedforward result with the previous output in a feedback loop.
- The architecture is implemented on FPGA using adders, shifters, and registers in a structured pipelined dataflow manner.
- The block diagram of the implemented retimed IIR filter is shown below.

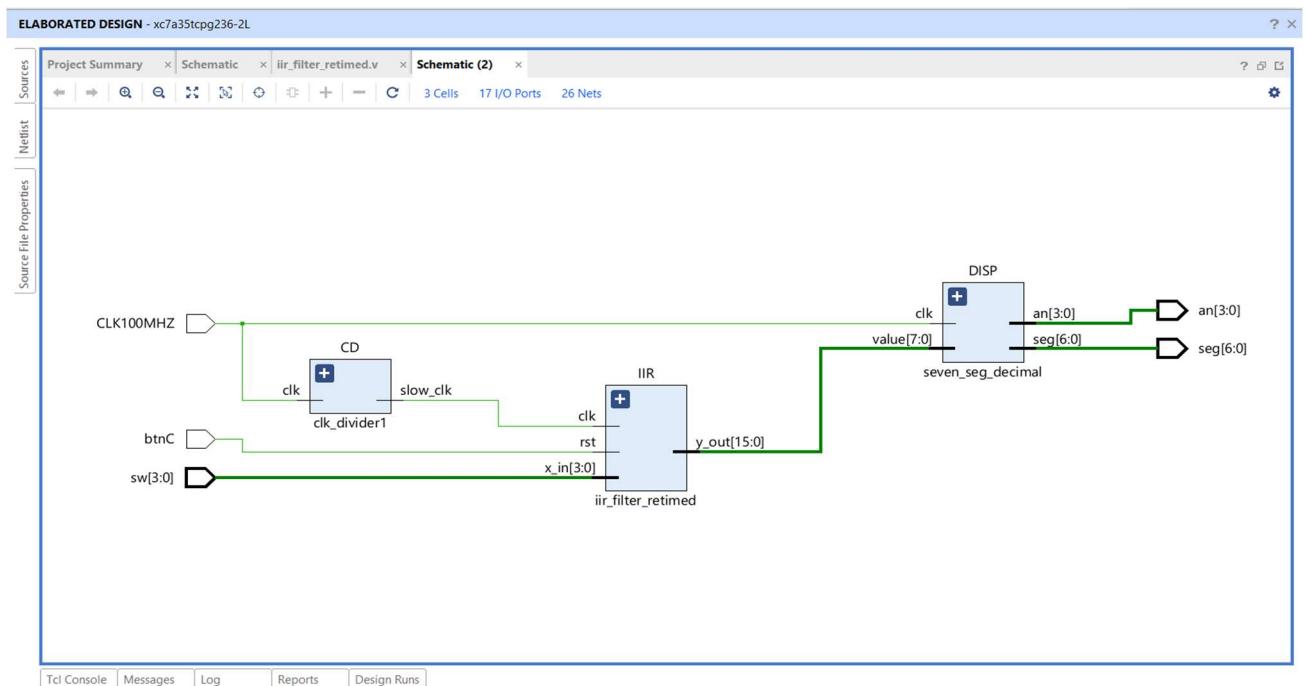


Fig 4.21) Elaborated Design of the Retimed IIR Filter Design

- **Comparison Between Non-Retimed and Retimed IIR Filter:**

Parameter	IIR Filter	Retimed IIR Filter
Critical Path Delay	3.664 ns <i>(higher)</i>	3.549 ns <i>(lower)</i>
Max Frequency	273 MHz <i>(lower)</i>	282 MHz <i>(higher)</i>
Latency	1 cycle <i>(lower)</i>	2 cycles <i>(higher)</i>
Throughput	1 output/cycle	1 output/cycle
LUT Utilization	59	67
SLICE registers	69 <i>(lower)</i>	87 <i>(higher)</i>
Dynamic Power	1 mW <i>(lower)</i>	2 mW <i>(higher)</i>

Conclusion:

- In this project, an IIR filter and its retimed architecture were designed and implemented on FPGA.
- The IIR filter was implemented using a recursive structure with feedback from previous outputs.
- The retimed IIR filter improves performance by redistributing registers in the forward path to reduce the critical path delay.
- The results show that the retimed design achieves better timing performance and higher maximum operating frequency compared to the normal IIR filter.
- However, this improvement leads to a slight increase in latency due to additional register stages in the forward path.
- The throughput of both designs remains the same, producing one output per clock cycle after pipeline filling.
- Overall, the comparison highlights the trade-off between timing performance and latency in digital filter design.

4.) Modeling and Simulation of Full Adder using Verilog HDL based on Combinational Logic Design

• Introduction:

- A full adder is a combinational circuit used to add three binary inputs: A, B, and Carry-in (Cin).
- It produces two outputs: Sum and Carry-out (Cout).
- It is a fundamental building block in arithmetic circuits such as adders and multipliers.
- The design is implemented using basic logic gates (XOR, AND, OR).
- In this project, the full adder is modeled and simulated using Verilog HDL.
- The functionality is verified by applying all possible input combinations.

• Direct implementation of Full Adder

- The full adder is implemented using combinational logic expressions.
- The Sum output is obtained using XOR operation of inputs A, B, and Cin.
- The Carry-out is generated using AND OR operations.
- The design directly maps the Boolean equations into hardware logic.
- No pipelining or sequential elements are used in this implementation.

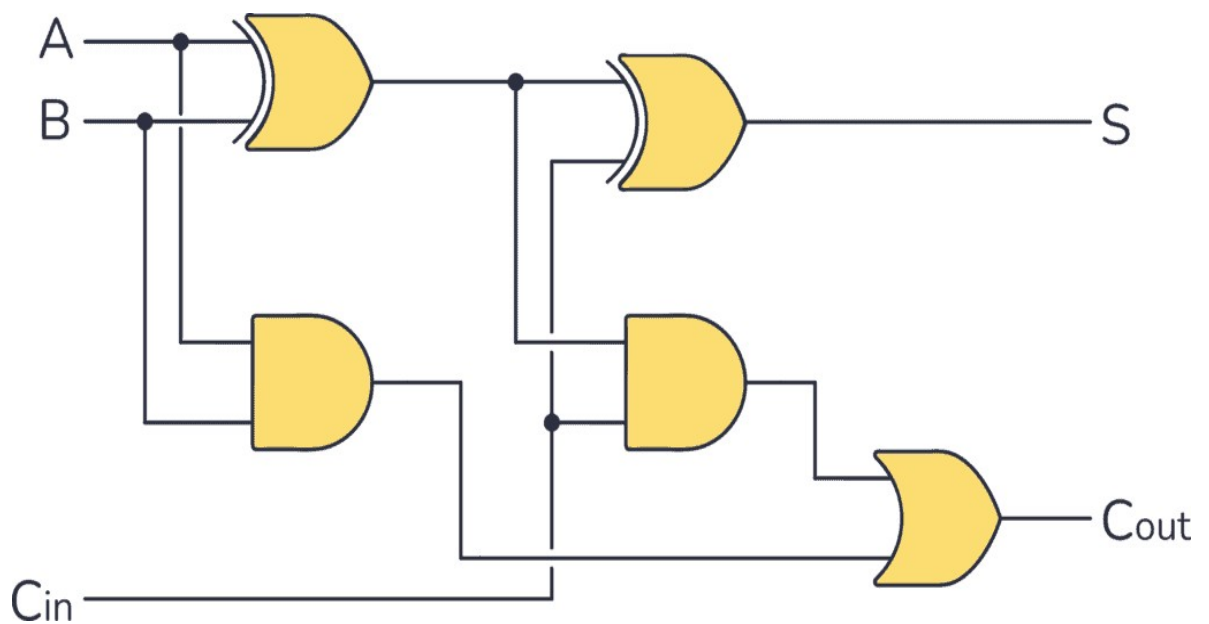


Fig 5.1) Full Adder Circuit Design

• **Truth Table and Output Representation of Full Adder:**

A	B	Cin	SUM	COUT	{COUT, SUM}	DECIMAL
0	0	0	0	0	00	0
0	0	1	1	0	01	1
0	1	0	1	0	01	1
0	1	1	0	1	10	2
1	0	0	1	0	01	1
1	0	1	0	1	10	2
1	1	0	0	1	10	2
1	1	1	1	1	11	3

- The table shows all 8 possible input combinations of A, B, and Cin.
- The Sum output is obtained using XOR operation of the three inputs.
- The Carry-out (Cout) is generated when at least two inputs are high.
- The combined output {Cout, Sum} represents the binary result of addition.
- The decimal column helps with easy verification of the output values.
- The results confirm the correct working of the full adder logic.

• Simulation of the Proposed Full Adder:

- The full adder design is simulated using Verilog HDL.
- A testbench is developed to apply all possible input combinations (A, B, Cin).
- The simulation is carried out using a digital simulation tool (Vivado/Model Sim).
- The waveform shows the relationship between inputs and outputs (Sum and Cout).
- The combined output {Cout, Sum} is also observed in the simulation.
- The results verify the correct functionality of the full adder for all cases.

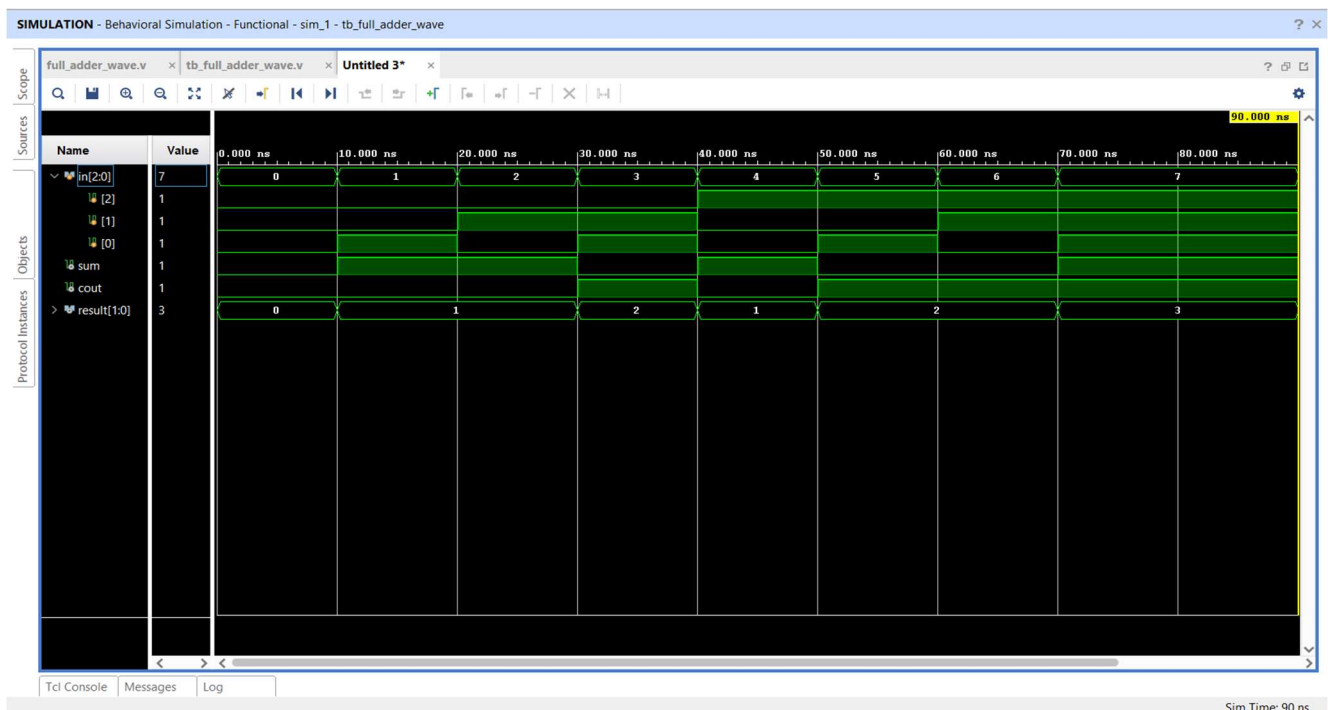


Fig 5.2) Simulation Results of Full Adder

• Output Results of Full Adder:

- The outputs Sum and Carry-out (Cout) are obtained for all input combinations.
- The simulation results match the theoretical truth table values.
- The combined output {Cout, Sum} correctly represents the binary addition result.
- The outputs are also displayed on the LCD (7-segment display) on the FPGA board.

- The values displayed correspond to the decimal equivalent of the binary output.
- The waveform confirms that the outputs changes immediately with input changes.
- The design is verified to be functionally correct and efficient.



Fig 5.3) Displayed Output for Input 000 (Cout, Sum)



Fig 5.4) Displayed Output for Input 001 (Cout, Sum)



Fig 5.5) Displayed Output for Input 010 (Cout, Sum)

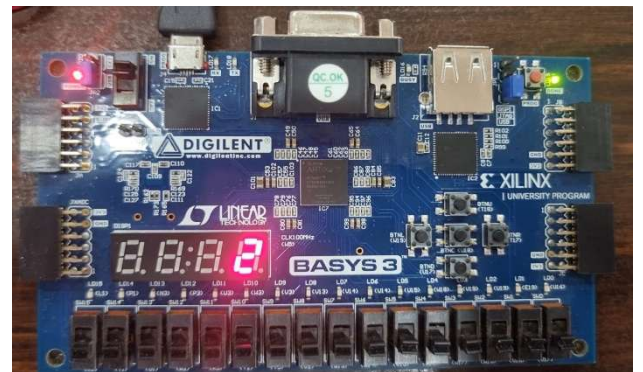


Fig 5.6) Displayed Output for Input 011 (Cout, Sum)

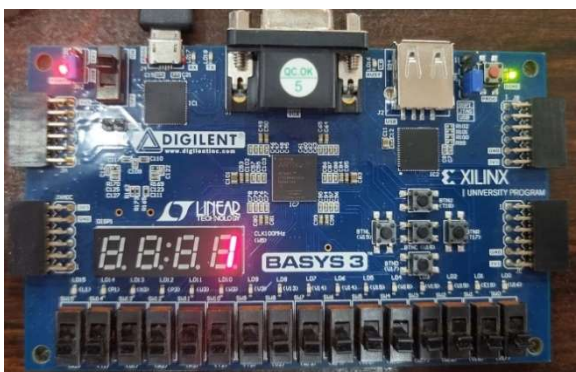


Fig 5.7) Displayed Output for Input 100 (Cout, Sum)

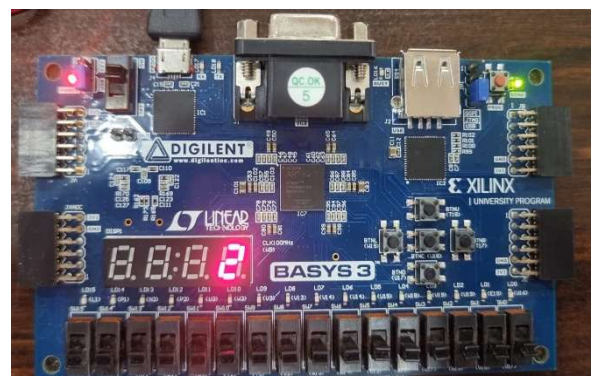


Fig 5.8) Displayed Output for Input 101 (Cout, Sum)

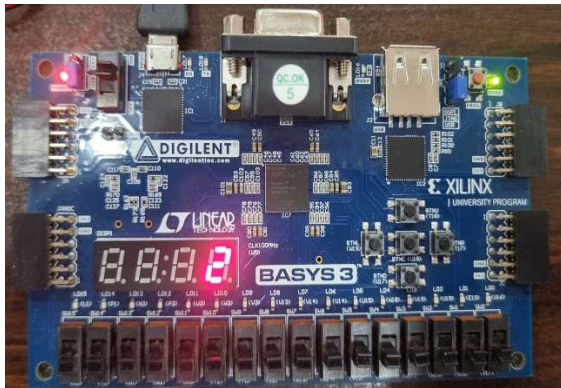


Fig 5.9) Displayed Output for Input 110 (Cout, Sum)

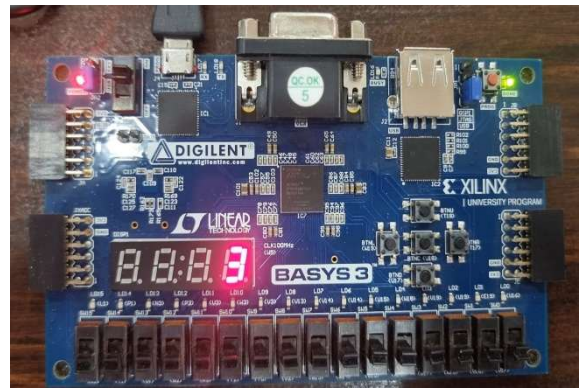


Fig 5.10) Displayed Output for Input 111 (Cout, Sum)

• **Timing Analysis of the Full Adder:**

- The timing analysis is performed using clock constraints in Vivado.
- The Worst Negative Slack (WNS) obtained is 6.636 ns, indicating no timing violations.
- The positive slack shows that the design operates within the specified clock period.
- The critical path is defined by the longest combinational path through XOR, AND, and OR gates.
- The design exhibits low propagation delay due to simple logic implementation.
- The results confirm that the full adder can operate at reliable speed on FPGA.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.636 ns	Worst Hold Slack (WHS): 0.258 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 53	Total Number of Endpoints: 53	Total Number of Endpoints: 28

All user specified timing constraints are met.

Fig 5.11) Design Timing Summary of Full Adder

Clock Summary				
Name	Waveform	Period (ns)	Frequency (MHz)	
CLK100MHZ	{0.000 5.000}	10.000	100.000	

Fig 5.12) Clock Period of Full Adder

Critical Path delay = Clock Period-WNS=10ns-6.636ns=3.364ns

Frequency = 1/Tcp=1/3.364ns=297.26 MHz

- **Resource Utilization of the Full Adder:**
- The full adder design utilizes LUTs (Look-Up Tables) for implementing combinational logic.
- A small number of flip-flops (registers) are used in the design.
- The overall resource utilization is low due to simple logic structure.
- The design occupies minimal area on the FPGA, making it efficient.
- Compared to the pipelined design, the number of flip-flops used is less.
- The Vivado report confirms efficient utilization of hardware resources.

Summary

Resource	Utilization	Available	Utilization %
LUT	12	20800	0.06
FF	30	41600	0.07
IO	13	106	12.26

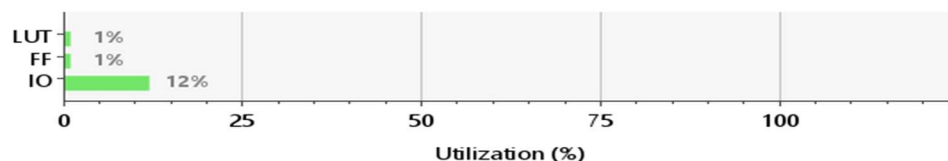


Fig 5.13) Overall Resource Utilization Summary of the Design

Q

≡

⬆

%

Hierarchy

Name	^1	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	Bonded IOB (106)	BUFGCTRL (32)
N top_basys3		12	30	13	12	13	1

Fig 5.14) Hierarchical Resource Utilization of the Top Module

- **Power Analysis of the Full Adder:**
 - The full adder design utilizes LUTs (Look-Up Tables) for implementing combinational logic.
 - A small number of flip-flops (registers) are used in the design.
 - The overall resource utilization is low due to simple logic structure.

- The design occupies minimal area on the FPGA, making it efficient.
- Compared to the pipelined design, the number of flip-flops used is less.
- The Vivado report confirms efficient utilization of hardware resources.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: **0.07 W**
Design Power Budget: **Not Specified**
Process: **typical**
Power Budget Margin: **N/A**
Junction Temperature: **25.3°C**
 Thermal Margin: 74.7°C (14.8 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power

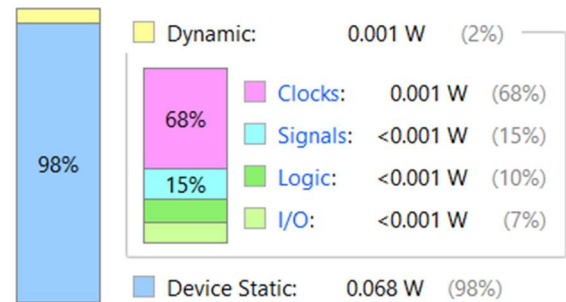


Fig 5.15) On-Chip Power Distribution of the Full Adder

Settings	Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Summary (0.07 W, Margin: N/A)	Vccint	1.000	0.011	0.001	0.010	Unspecified	NA
Power Supply	Vccaux	1.800	0.013	0.000	0.013	Unspecified	NA
Utilization Details	Vcco33	3.300	0.000	0.000	0.000	Unspecified	NA
Hierarchical (0.001 W)	Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Clocks (0.001 W)	Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Signals (<0.001 W)	Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Data (<0.001 W)	Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Set/Reset (<0.001 W)	Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Logic (<0.001 W)	Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
I/O (<0.001 W)	Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTA Vcc	1.000	0.000	0.000	0.000	Unspecified	NA
	MGTA Vtt	1.200	0.000	0.000	0.000	Unspecified	NA
	Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

Fig 5.16) Power Supply Breakdown of the Full Adder

- **Elaborated Design of the Hardware Circuit of Full Adder:**

- The hardware design of the full adder is based on combinational logic operations.
- The inputs A, B, and Cin are applied to the circuit to perform binary addition.
- The Sum output is generated using XOR logic, while Carry-out (Cout) is generated using AND OR logic.
- The design is implemented using LUTs and registers on FPGA.
- The outputs are combined as {Cout, Sum} to represent the result.
- The output is displayed on the 7-segment display of the Basys 3 FPGA board.
- The overall architecture is simple, efficient, and suitable for basic arithmetic operations.

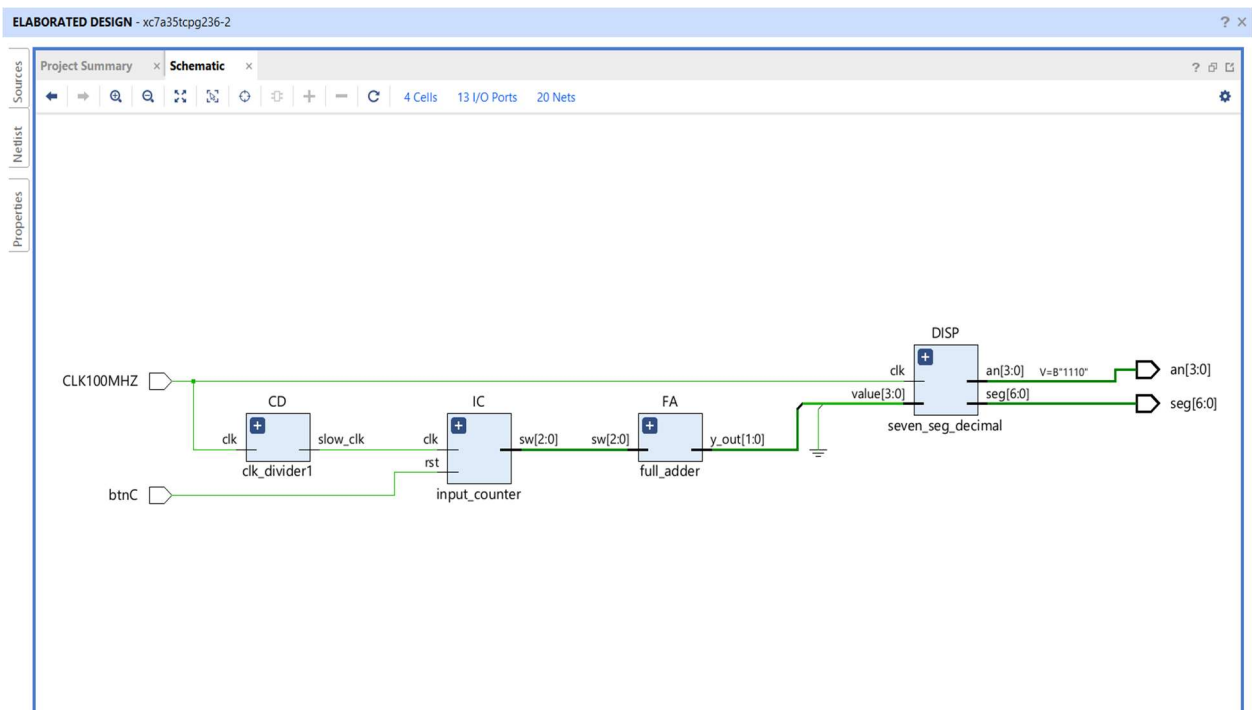


Fig 5.17) Elaborated Design of the Full Adder Design

- **Conclusion:**

- In this project, a full adder was successfully designed and implemented using Verilog HDL.
- The design was verified through simulation and FPGA implementation on Basys 3.
- The output obtained matched the expected theoretical results for all input combinations.
- The timing and resource analysis showed efficient performance with low hardware utilization.

5.) Modeling and Simulation of Pipelined Full Adder using Verilog HDL based on Pipelining Technique

• Introduction:

- A pipelined full adder is an improved version of the full adder using the pipelining technique.
- The computation is divided into multiple stages using registers.
- This reduces the critical path delay and improves operating speed.
- The design introduces latency but increases overall throughput.
- In this project, the pipelined full adder is modeled using Verilog HDL.
- The functionality is verified through simulation and FPGA implementation.

• Input–Output Table of Pipelined Full Adder:

- $s1 = A \oplus B$
- $c1 = A \cdot B$
- $sum_s = OLD\ s1 \oplus OLD\ cin_r$
- $cout_s = OLD\ c1 + (OLD\ s1 \cdot OLD\ cin_r)$

Clock	Input (A, B, Cin)	Stage1(s1, c1)	Stage2 Output (Cout, Sum)
1	000	00	-
2	001	00	-
3	010	10	00(for 000)
4	011	10	01(for 001)
5	100	10	01(for 010)
6	101	10	10(for 011)
7	110	01	01(for 100)
8	111	01	10(for 101)
9	-	-	10(for 110)
10	-	-	11(for 111)

Simulation of the Proposed Pipelined Full Adder:

- The pipelined full adder is simulated using Verilog HDL.
- A testbench is used to apply all input combinations sequentially.
- The simulation is carried out using Vivado/Model Sim waveform viewer.
- The outputs (Sum and Cout) are observed along with the combined output {Cout, Sum}.
- Due to pipelining, the output is obtained with a delay of two clock cycles.
- Initial outputs are invalid during pipeline filling stage.
- After filling, the circuit produces one output per clock cycle.
- The simulation results match the expected full-adder output with delay.



Fig 6.1) Simulation Results of Pipelined Full Adder

• Output Results of Full Adder:

- The outputs Sum and Carry-out (Cout) are obtained correctly for all input combinations.
- The combined output {Cout, Sum} represents the final binary addition result.
- The outputs are observed with a delay of two clock cycles due to pipelining.
- Initial outputs are invalid during the pipeline filling stage.
- After the initial delay, the circuit produces continuous outputs for every clock cycle.
- The results match the normal full-added outputs with a fixed delay.
- The outputs are also displayed on the 7-segment display of the Basys 3 FPGA board.



Fig 6.2) Displayed Output for Input 000

(Clock Cycle 1 – Pipeline Filling)



Fig 6.3) Displayed Output for Input 001

(Clock Cycle 2 – Pipeline Filling)

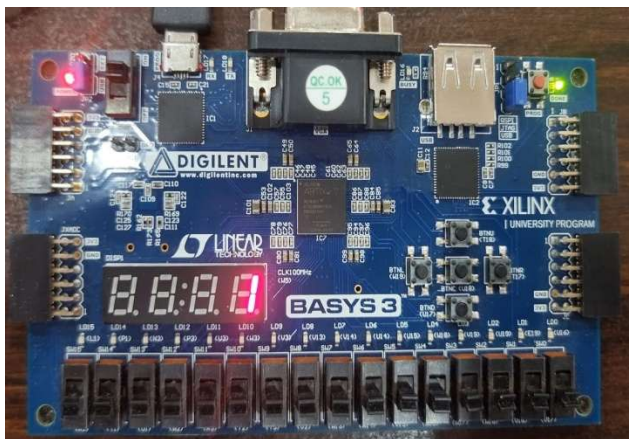


Fig 6.4) Displayed Output for Input 011 (Clock Cycle 4)
Cycle 5)



Fig 6.5) Displayed Output for Input 100 (Clock

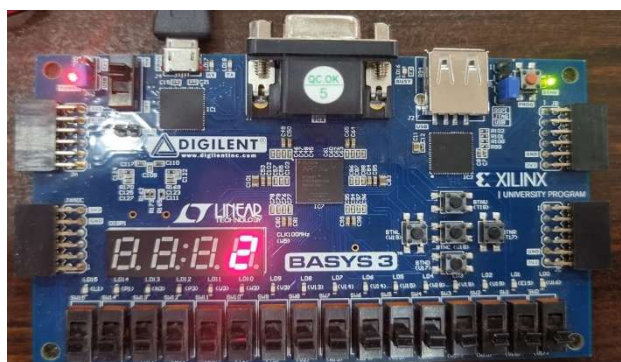


Fig 6.6) Displayed Output for Input 101 (Clock Cycle 6)
7)



Fig 6.7) Displayed Output for Input 110 (Clock Cycle



Fig 6.8) Displayed Output for Input 111 (Clock Cycle 8)

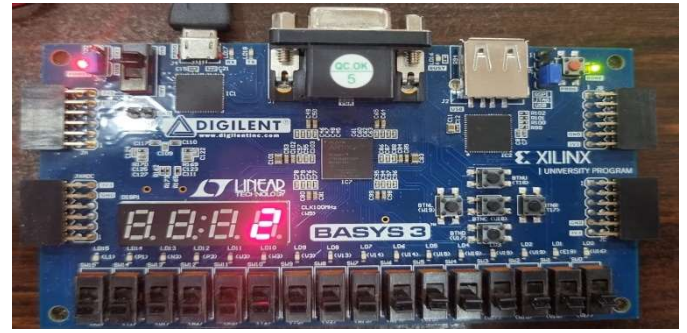


Fig 6.9) Displayed Output for Input (Clock Cycle 9)

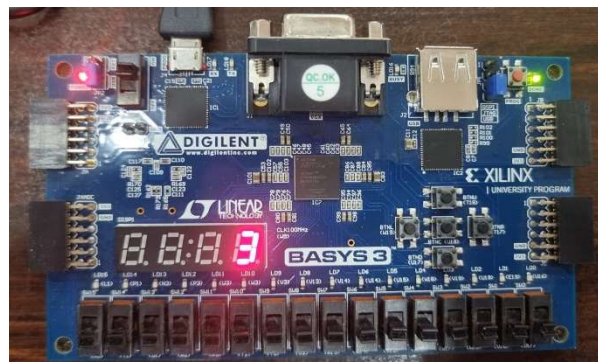


Fig 6.10) Displayed Output for Input Combination (-) (Clock Cycle 10)

• **Timing Analysis of the Pipelined Full Adder:**

- The timing analysis is performed using clock constraints in Vivado.
- The pipelined design reduces the critical path delay by dividing the logic into stages.
- Each pipeline stage contains simpler logic, resulting in faster operation.
- The Worst Negative Slack (WNS) is positive, indicating no timing violations.
- Compared to the normal full adder, the pipelined design achieves improved timing performance.
- The design supports higher operating frequency due to reduced delay per stage.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.733 ns	Worst Hold Slack (WHS): 0.259 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 53	Total Number of Endpoints: 53	Total Number of Endpoints: 28

All user specified timing constraints are met.

Fig 6.11) Design Timing Summary of Pipelined Full Adder

Clock Summary			
Name	Waveform	Period (ns)	Frequency (MHz)
CLK100MHZ	{0.000 5.000}	10.000	100.000

Fig 6.12) Clock Period of Full Adder

Critical Path delay = Clock Period-WNS=10ns-6.733ns=3.267ns

Frequency = $1/T_{cp}=1/3.364ns=306.09$ MHz

- **Resource Utilization of the pipelined Full Adder:**

- The pipelined full adder utilizes LUTs (Look-Up Tables) for implementing combinational logic.
- Additional flip-flops (registers) are used to implement pipeline stages.
- The number of flip-flops is higher compared to the normal full adder due to stage-wise storage.
- The overall resource utilization is slightly increased due to pipelining.
- The design still occupies low area on the FPGA, ensuring efficient implementation.
- The Vivado report confirms optimized utilization of hardware resources.

Summary

Resource	Utilization	Available	Utilization %
LUT	13	20800	0.06
FF	37	41600	0.09
IO	13	106	12.26

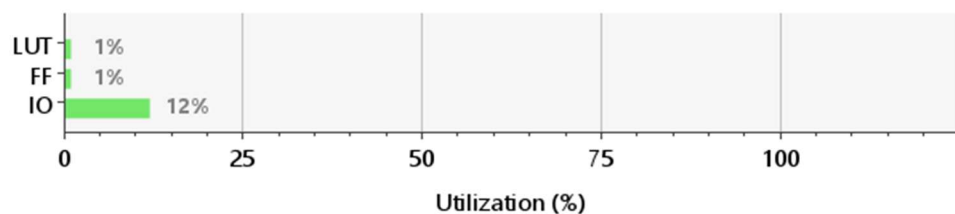


Fig 6.13) Overall Resource Utilization Summary of the Design

Hierarchy					
Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (106)	BUFGCTRL (32)	
top_basys3	13	37	13	1	

Fig 6.14) Hierarchical Resource Utilization of the Top Module

• **Power Analysis of the Pipelined Full Adder:**

- The power analysis is performed using the Vivado power report.
- The total power consumption includes dynamic power and static (leakage) power.
- The dynamic power is slightly higher due to increased switching activity from pipeline registers.
- The use of additional flip-flops contributes to higher power consumption compared to the normal full adder.
- Static power remains relatively low due to efficient resource utilization.
- Overall power consumption is moderate and acceptable for high-speed applications.

Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power: 0.089 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 25.4°C
 Thermal Margin: 59.6°C (11.8 W)
 Ambient Temperature: 25.0 °C
 Effective θ_{JA} : 5.0°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Medium

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

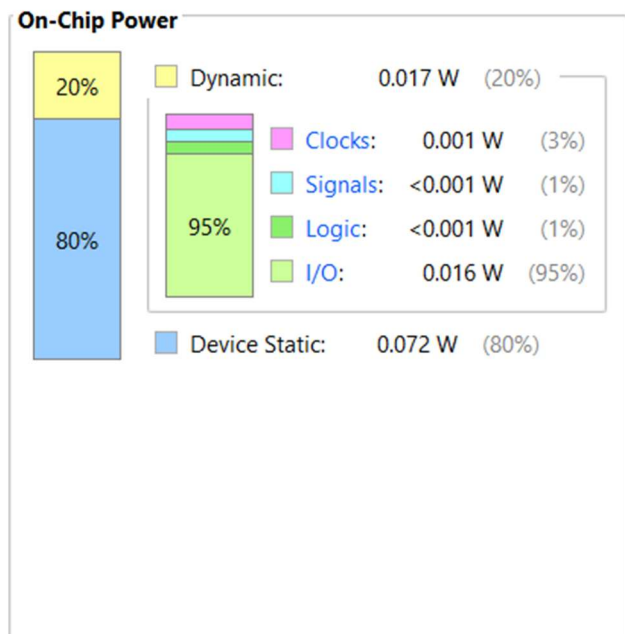


Fig 6.15) On-Chip Power Distribution of the Pipelined Full Adder

Supply Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Vccint	1.000	0.011	0.001	0.010	Unspecified	NA
Vccaux	1.800	0.013	0.001	0.013	Unspecified	NA
Vcco33	3.300	0.006	0.005	0.001	Unspecified	NA
Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
MGTAVcc	1.000	0.000	0.000	0.000	Unspecified	NA
MGTAVtt	1.200	0.000	0.000	0.000	Unspecified	NA
Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

Fig 6.16) Power Supply Breakdown of the Pipelined Full Adder

- **Elaborated Design of the Hardware Circuit of Pipelined Full Adder:**

- The pipelined full adder is designed by dividing the computation into multiple stages using registers.
- The first stage computes partial results such as $A \oplus B$ and $A \cdot B$.
- These intermediate values are stored in pipeline registers.
- In the next stage, the final outputs Sum and Carry-out (Cout) are computed using stored values and Cin.
- The use of registers between stages enables parallel processing of multiple input combinations.
- The outputs are combined as {Cout, Sum} to represent the final result.
- The output is displayed on the 7-segment display of the Basys 3 FPGA board.
- The design improves speed by reducing critical path delay, at the cost of additional hardware and latency.

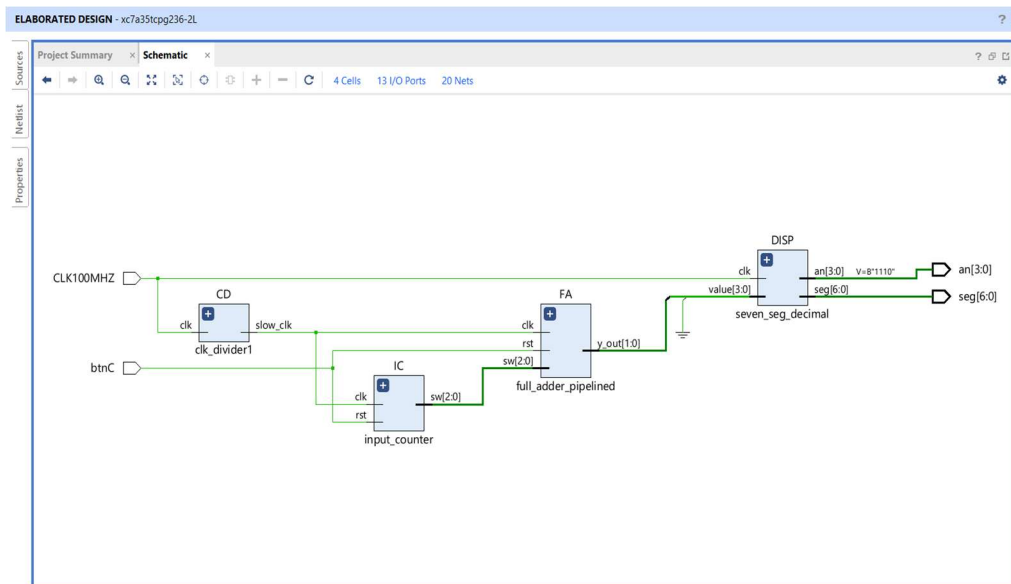


Fig 6.17) Elaborated Design of the Pipelined Full Adder Design

- **Comparison Between Full adder and Pipelined Full Adder :**

Parameter	Full Adder	Pipelined Full Adder
Critical Path Delay	3.364 ns <i>(higher)</i>	3.267 ns <i>(lower)</i>
Max Frequency	297 MHz <i>(lower)</i>	306 MHz <i>(higher)</i>
Latency	1 cycle <i>(lower)</i>	2 cycles <i>(higher)</i>
Throughput	1 output/cycle	1 output/cycle
LUT Utilization	12	13
SLICE registers	30 <i>(lower)</i>	37 <i>(higher)</i>
Dynamic Power	1 mW <i>(lower)</i>	17 mW <i>(higher)</i>

- **Conclusion:**

- Designed and compared Full Adder and Pipelined Full Adder.
- Full Adder is simple but has higher delays.
- Pipelining reduces critical path and improves speed.
- Throughput remains almost the same.
- Latency increases in pipelined design.
- Pipelined design is better for high-speed applications.